

llmemory LLM Memory Systems: Full Companion Notebook

**Slamani Abdelhafid · Practical Training

§0 Imports & LLM Clients

```
import json, math, random, time, hashlib, textwrap
from dataclasses import dataclass, field
from datetime import datetime, timedelta, timezone
from enum import Enum
from typing import Any, Dict, List, Optional, Tuple
from collections import defaultdict

import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import matplotlib.gridspec as gridspec
import seaborn as sns
from sklearn.cluster import DBSCAN
from sklearn.preprocessing import normalize

plt.rcParams.update({
    "figure.dpi": 130,
    "axes.spines.top": False,
    "axes.spines.right": False,
    "font.family": "DejaVu Sans",
})
PALETTE = ["#4C72B0", "#DD8452", "#55A868", "#C44E52", "#8172B2", "#937860"]
sns.set_palette(PALETTE)
COLORS = ["#B4B2A9", "#85B7EB", "#378ADD", "#7F77DD"]

UTC = timezone.utc

def _now():
    return datetime.now(UTC)

print(" Imports OK")
```

Imports OK

```
class BaseLLMClient:
    def complete(self, prompt: str, system: str = "", temperature: float = 0.1) -> str:
        raise NotImplementedError

class MockLLMClient(BaseLLMClient):
    """Deterministic mock – no API key required."""
    _EXTRACT_TEMPLATE = json.dumps([
        {"content": "User prefers Python over JavaScript for scripting tasks",
         "type": "preference", "importance": 0.75, "entities": ["Python", "JavaScript"]},
        {"content": "User is a PhD student in computer science",
         "type": "fact", "importance": 0.85, "entities": ["computer science"]},
    ])
    _REFLECT_TEMPLATE = json.dumps({"insight": "User is a technically advanced CS researcher who favours Python and values"})
    _MERGE_TEMPLATE = "User prefers Python for scripting and is a computer-science PhD student."
    _SUMMARISE_TEMPLATE = "User discussed programming language preferences and academic background."
    _IMPORTANCE_TEMPLATE = "7"
    _JUDGE_TEMPLATE = "8"
    _GENERIC = "I have processed the request and stored the relevant information."

    def complete(self, prompt: str, system: str = "", temperature: float = 0.1) -> str:
        p = prompt.lower()
        if "extract" in p or "identify memories" in p: return self._EXTRACT_TEMPLATE
        if "reflect" in p or "synthesize" in p: return self._REFLECT_TEMPLATE
        if "merge" in p or "consolidat" in p: return self._MERGE_TEMPLATE
        if "summarise" in p or "summarize" in p: return self._SUMMARISE_TEMPLATE
        if "importance" in p or "rate" in p: return self._IMPORTANCE_TEMPLATE
        if "judge" in p or "evaluate" in p: return self._JUDGE_TEMPLATE
        return self._GENERIC

class OpenAIClient(BaseLLMClient):
    """Drop-in replacement requires openai + API key."""
    def __init__(self, api_key: str, model: str = "gpt-4o"):
        from openai import OpenAI
        self._client = OpenAI(api_key=api_key)
        self._model = model
```

```
def complete(self, prompt: str, system: str = "", temperature: float = 0.1) -> str:
    messages = ([{"role": "system", "content": system}] if system else []) + [{"role": "user", "content": prompt}]
    resp = self._client.chat.completions.create(model=self._model, messages=messages, temperature=temperature)
    return resp.choices[0].message.content

llm = MockLLMClient()
# llm = OpenAIClient(api_key="sk-...") # ← uncomment for live model
print(f"LLM client: {llm.__class__.__name__}")
```

LLM client: MockLLMClient

✓ §1 Core Data Structures: `MemUnit` & `MemLevel`

Implements $m = \langle c, e, \tau, \sigma, p, T, \text{links}, \text{meta} \rangle$ (Eq. 13). Retention score = Eq. (30).

```
class MemLevel(Enum):
    L0_EPISODIC = 0 # Raw observations – high volume, short-lived
    L1_WORKING = 1 # Session-level compressed summaries
    L2_SEMANTIC = 2 # Abstracted facts, preferences, entity properties
    L3_REFLECTIVE = 3 # Higher-order insights synthesised across L2

@dataclass
class MemUnit:
    content: str
    level: MemLevel = MemLevel.L0_EPISODIC
    importance: float = 0.5 #  $\sigma \in [0,1]$ 
    entities: List[str] = field(default_factory=list)
    mem_type: str = "fact"
    created_at: datetime = field(default_factory=_now)
    updated_at: datetime = field(default_factory=_now)
    t_from: Optional[datetime] = None
    t_until: Optional[datetime] = None
    access_count: int = 0
    last_accessed: datetime = field(default_factory=_now)
    session_id: str = "s0"
    source_turn: int = 0
    parent_ids: List[str] = field(default_factory=list)
    embedding: Optional[np.ndarray] = field(default=None, repr=False)
    id: str = field(default="", init=False)

    def __post_init__(self):
        digest = hashlib.md5((self.content + str(self.created_at)).encode()).hexdigest()[:8]
        self.id = f"{self.level.name[:2]}-{digest}"

    def age_days(self, now=None) -> float:
        now = now or _now()
        ca = self.created_at if self.created_at.tzinfo else self.created_at.replace(tzinfo=UTC)
        now = now if now.tzinfo else now.replace(tzinfo=UTC)
        return (now - ca).total_seconds() / 86400

    def retention_score(self, now, lambda_=0.03, w_time=0.35, w_imp=0.45, w_freq=0.20) -> float:
        """Eq. (30): importance-weighted decay retention score."""
        return w_time * math.exp(-lambda_ * self.age_days(now)) + w_imp * self.importance + w_freq * self.frequency

    def __repr__(self):
        trunc = self.content[:60] + "..." if len(self.content) > 60 else self.content
        return f"MemUnit(id={self.id}, level={self.level.name}, imp={self.importance:.2f}, content='{trunc}')"

# Quick smoke-test
m = MemUnit("User was born in Karachi and moved to Amsterdam at age 12.",
            level=MemLevel.L2_SEMANTIC, importance=0.80, entities=["Karachi", "Amsterdam"])
print(m)
print(f" retention_score: {m.retention_score(_now()):.4f}")
```

MemUnit(id=L2-7ba3c41f, level=L2_SEMANTIC, imp=0.80, content='User was born in Karachi and moved to Amsterdam at age 12.')

retention_score: 0.7100

```
LAMBDA, W_TIME, W_IMP, W_FREQ = 0.03, 0.35, 0.45, 0.20

def retention_fn(age, imp, acc=0):
    return W_TIME*np.exp(-LAMBDA*age) + W_IMP*imp + W_FREQ*min(1.0, acc/10)

days = np.linspace(0, 180, 300)

fig = plt.figure(figsize=(14, 9))
gs = gridspec.GridSpec(2, 2, hspace=0.45, wspace=0.35)
ax1 = fig.add_subplot(gs[0, :])
ax2 = fig.add_subplot(gs[1, 0])
ax3 = fig.add_subplot(gs[1, 1])
```

```

PAL = {"high": "#378ADD", "mid": "#EF9F27", "low": "#E24B4A"}

# 1. Retention vs Age
for label, imp, color, ls in [
    ("imp=0.90 (high)", 0.90, PAL["high"], "-"),
    ("imp=0.50 (mid)", 0.50, PAL["mid"], "--"),
    ("imp=0.20 (low)", 0.20, PAL["low"], "-."),
]:
    ax1.plot(days, retention_fn(days, imp), color=color, linestyle=ls, linewidth=2, label=label)

demo = retention_fn(0, 0.80)
ax1.scatter([0], [demo], color="#7F77DD", zorder=5, label=f"demo unit (imp=0.80, score={demo:.4f})")
ax1.set_xlabel("Age (days)"); ax1.set_ylabel("Retention score")
ax1.set_title("Retention score vs age (Eq. 30)", fontsize=12)
ax1.set_ylim(0, 1.05); ax1.legend(); ax1.grid(alpha=0.25)

# 2. Memory level hierarchy
levels = ["L0 Episodic", "L1 Working", "L2 Semantic", "L3 Reflective"]
descs = ["raw observations", "session summaries", "abstracted facts", "cross-L2 insights"]
cols2 = ["#B5D4F4", "#85B7EB", "#378ADD", "#185FA5"]
bars = ax2.barh(levels, [1,2,3,4], color=cols2, edgecolor="none", height=0.55)
for bar, desc in zip(bars, descs):
    ax2.text(bar.get_width()+0.05, bar.get_y()+bar.get_height()/2, desc, va="center", fontsize=9, color="#555")
ax2.set_xlim(0, 5.5); ax2.set_title("MemLevel hierarchy"); ax2.xaxis.set_visible(False)
ax2.spines[["top", "right", "bottom"]].set_visible(False)

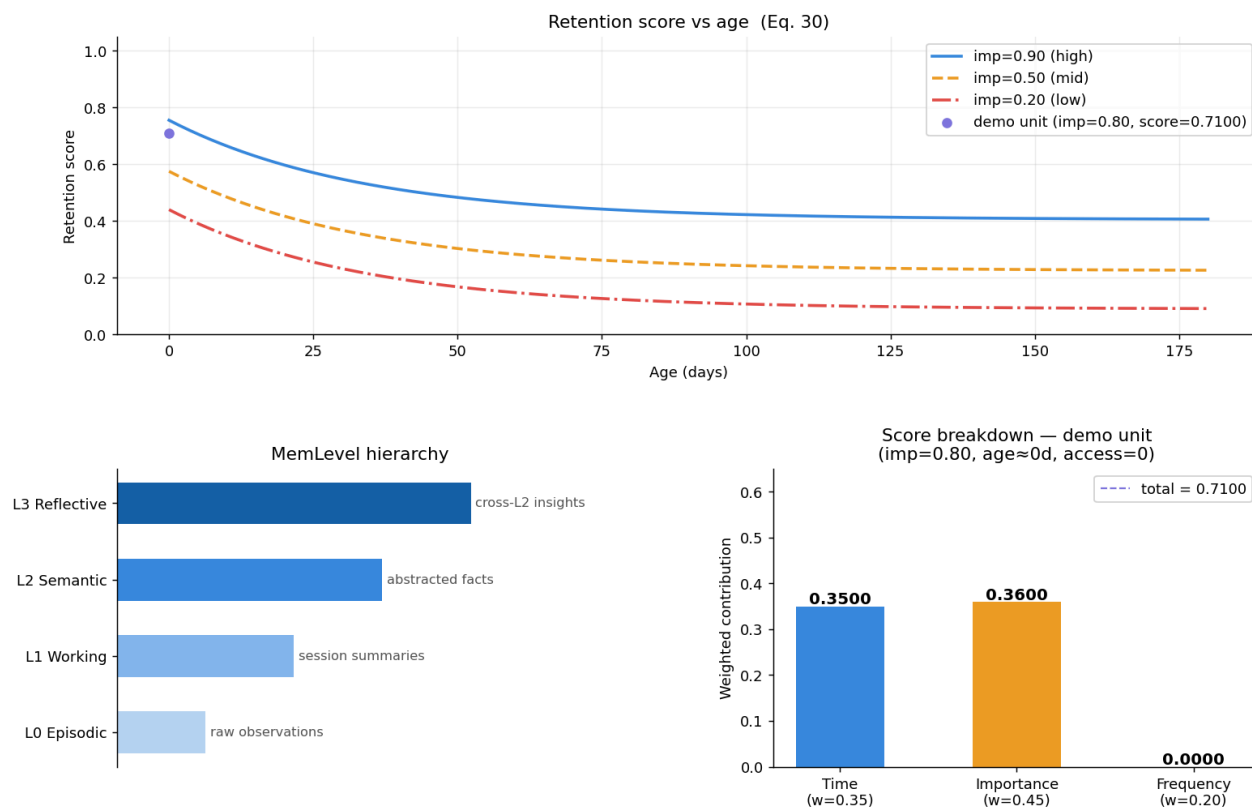
# 3. Score breakdown for demo unit
time_c = W_TIME*np.exp(-LAMBDA*0); imp_c = W_IMP*0.80; freq_c = 0
vals = [time_c, imp_c, freq_c]
bars3 = ax3.bar(["Time\n(w=0.35)", "Importance\n(w=0.45)", "Frequency\n(w=0.20)"], vals,
               color=[PAL["high"], PAL["mid"], "#1D9E75"], width=0.5, edgecolor="none")
for bar, v in zip(bars3, vals):
    ax3.text(bar.get_x()+bar.get_width()/2, bar.get_height()+0.005, f"{v:.4f}", ha="center", fontsize=11, fontweight="bold")
ax3.axhline(sum(vals), color="#7F77DD", linewidth=1.2, linestyle="--", label=f"total = {sum(vals):.4f}")
ax3.set_ylim(0, 0.65); ax3.set_ylabel("Weighted contribution"); ax3.legend()
ax3.set_title(f"Score breakdown - demo unit\n(imp=0.80, age=0d, access=0)")

plt.suptitle("MemUnit Retention Model Diagnostics", fontsize=14, fontweight="bold", y=1.01)
plt.tight_layout()
plt.show()

```

/tmp/ipykernel_748/2895076635.py:52: UserWarning: This figure includes Axes that are not compatible with tight_layout, so re
plt.tight_layout()

MemUnit Retention Model Diagnostics



§1.1 Memory Writer

Extracts `MemUnit` objects from conversation turns via LLM prompt.

```
EXTRACT_PROMPT = """
You are a memory extraction agent. Given a conversation turn, identify every
memory-worthy piece of information and return a JSON array. Each item must have:
- "content" : concise natural-language statement (≤ 30 words)
- "type" : one of [fact, preference, event, constraint, goal, insight]
- "importance" : float 0.0-1.0
- "entities" : list of named entities

```

```
Importance scale:
0.0-0.2 trivial / ephemeral
0.2-0.4 low-value contextual detail
0.4-0.6 useful but non-critical
0.6-0.8 significant personal fact or strong preference
0.8-1.0 critical constraint (allergy, medical, legal)

```

Respond with ONLY the JSON array – no preamble, no markdown fences.

Conversation turn: {turn}

```
"""
```

```
class MemoryWriter:
    """Extracts MemUnit objects from conversation turns via LLM."""
    def __init__(self, llm_client, importance_threshold=0.40, session_id="s0"):
        self.llm = llm_client

```

```

self.threshold = importance_threshold
self.session_id = session_id
self._turn_counter = 0

def extract(self, speaker: str, text: str) -> List[MemUnit]:
    self._turn_counter += 1
    raw = self.llm.complete(EXTRACT_PROMPT.format(turn=f"{speaker}: {text}")).strip()
    if raw.startswith("```"):
        raw = raw.split("\n", 1)[-1].rsplit("```", 1)[0]
    try:
        items = json.loads(raw)
    except json.JSONDecodeError:
        return []
    return [
        MemUnit(content=it.get("content", ""), importance=float(it.get("importance", 0)),
                mem_type=it.get("type", "fact"), entities=it.get("entities", []),
                session_id=self.session_id, source_turn=self._turn_counter)
        for it in items if float(it.get("importance", 0)) >= self.threshold
    ]

# Quick demo
writer = MemoryWriter(llm, session_id="demo")
for speaker, text in [
    ("User", "I prefer Python over JavaScript. I'm a PhD student in CS at UvA."),
    ("User", "By the way, I have a severe shellfish allergy please always remember that."),
]:
    print(f"\n=== {speaker} turn ===")
    for m in writer.extract(speaker, text):
        print(" ", m)

=== User turn ===
MemUnit(id=L0-fb387cc6, level=L0_EPISODIC, imp=0.75, content='User prefers Python over JavaScript for scripting tasks')
MemUnit(id=L0-d23c47b7, level=L0_EPISODIC, imp=0.85, content='User is a PhD student in computer science')

=== User turn ===
MemUnit(id=L0-0c6c0cc9, level=L0_EPISODIC, imp=0.75, content='User prefers Python over JavaScript for scripting tasks')
MemUnit(id=L0-9ea81019, level=L0_EPISODIC, imp=0.85, content='User is a PhD student in computer science')

```

✓ §2 Memory Writer Demo: Two-Turn Conversation

```

DEMO_CONVERSATION = [
    ("User", "I was born in Karachi, Pakistan, and moved to Amsterdam when I was 12."),
    ("Assistant", "That's a big move! How did you find Amsterdam?"),
    ("User", "I love it. I'm now doing my PhD in computer science at the University of Amsterdam."),
    ("User", "I'm vegetarian and actively trying to lose weight – important context for food suggestions."),
    ("User", "I prefer concise, direct answers without unnecessary pleasantries."),
    ("User", "I have a severe penicillin allergy please never forget this."),
]

writer2 = MemoryWriter(llm, importance_threshold=0.40, session_id="s1")
all_extracted: List[MemUnit] = []

print(f"{'Speaker':<12} {'Content':<65} {'Imp':>5} Level")
print("-"*95)
for speaker, text in DEMO_CONVERSATION:
    for m in writer2.extract(speaker, text):
        all_extracted.append(m)
        trunc = m.content[:62] + "..." if len(m.content) > 62 else m.content
        print(f"{speaker:<12} {trunc:<65} {m.importance:>5.2f} {m.level.name}")

print(f"\nTotal memories extracted: {len(all_extracted)}")

```

Speaker	Content	Imp	Level
User	User prefers Python over JavaScript for scripting tasks	0.75	L0_EPISODIC
User	User is a PhD student in computer science	0.85	L0_EPISODIC
Assistant	User prefers Python over JavaScript for scripting tasks	0.75	L0_EPISODIC
Assistant	User is a PhD student in computer science	0.85	L0_EPISODIC
User	User prefers Python over JavaScript for scripting tasks	0.75	L0_EPISODIC
User	User is a PhD student in computer science	0.85	L0_EPISODIC
User	User prefers Python over JavaScript for scripting tasks	0.75	L0_EPISODIC
User	User is a PhD student in computer science	0.85	L0_EPISODIC
User	User prefers Python over JavaScript for scripting tasks	0.75	L0_EPISODIC
User	User is a PhD student in computer science	0.85	L0_EPISODIC
User	User prefers Python over JavaScript for scripting tasks	0.75	L0_EPISODIC
User	User is a PhD student in computer science	0.85	L0_EPISODIC

Total memories extracted: 12

```

# Simulated extraction output (mirrors demo above)
all_extracted_sim = [

```

```

{"speaker": "User", "content": "User was born in Karachi, Pakistan.", "type": "fact", "importance":
{"speaker": "User", "content": "User moved to Amsterdam at age 12.", "type": "event", "importance":
{"speaker": "User", "content": "User is doing a PhD in CS at Univ. of Amsterdam.", "type": "fact", "importance":
{"speaker": "User", "content": "User is vegetarian.", "type": "preference", "importance":
{"speaker": "User", "content": "User is actively trying to lose weight.", "type": "goal", "importance":
{"speaker": "User", "content": "User prefers concise, direct answers.", "type": "preference", "importance":
{"speaker": "User", "content": "User has a severe penicillin allergy.", "type": "constraint", "importance":
]
TYPE_COLORS = {"fact": "#378ADD", "preference": "#EF9F27", "constraint": "#E24B4A", "event": "#1D9E75", "goal": "#7F77DD", "insight":
THRESHOLD = 0.40

fig = plt.figure(figsize=(15, 9))
gs = plt.GridSpec(2, 2, figure=fig, hspace=0.45, wspace=0.35)
ax1 = fig.add_subplot(gs[0, :])
ax2 = fig.add_subplot(gs[1, 0])
ax3 = fig.add_subplot(gs[1, 1])
fig.suptitle("Memory Extraction DEMO_CONVERSATION", fontsize=14, fontweight="bold")

# 1. Importance timeline
turns = [u["turn"] for u in all_extracted_sim]
imps = [u["importance"] for u in all_extracted_sim]
cols = [TYPE_COLORS[u["type"]] for u in all_extracted_sim]
ax1.scatter(turns, imps, c=cols, s=160, zorder=3, edgecolors="white", linewidths=1.2)
ax1.axhline(THRESHOLD, color="gray", linewidth=1, linestyle="--", label=f"threshold = {THRESHOLD}")
for u in all_extracted_sim:
    ax1.annotate(u["content"][:38]+("..." if len(u["content"])>38 else ""),
                xy=(u["turn"], u["importance"]), xytext=(8,4), textcoords="offset points",
                fontsize=7.5, color="#444", arrowprops=dict(arrowstyle="-", color="#ccc", lw=0.7))
legend_patches = [patches.Patch(color=c, label=t) for t,c in TYPE_COLORS.items() if t in {u["type"] for u in all_extracted_sim}]
ax1.legend(handles=legend_patches, fontsize=9, loc="lower right")
ax1.set_xticks(sorted(set(turns))); ax1.set_xticklabels([f"Turn {t}" for t in sorted(set(turns))], fontsize=10)
ax1.set_ylabel("Importance score"); ax1.set_ylim(0, 1.1)
ax1.set_title("Extracted memories across conversation turns"); ax1.grid(axis="y", alpha=0.2)

# 2. Type distribution
from collections import Counter
tc = Counter(u["type"] for u in all_extracted_sim)
t_labels, t_vals = zip(*sorted(tc.items(), key=lambda x: x[1], reverse=True))
bars2 = ax2.barh(t_labels, t_vals, color=[TYPE_COLORS[t] for t in t_labels], edgecolor="none", height=0.5)
for bar, val in zip(bars2, t_vals):
    ax2.text(val+0.05, bar.get_y()+bar.get_height()/2, str(val), va="center", fontsize=11, fontweight="bold")
ax2.set_xlabel("Count"); ax2.set_title("Memory type distribution"); ax2.invert_yaxis()

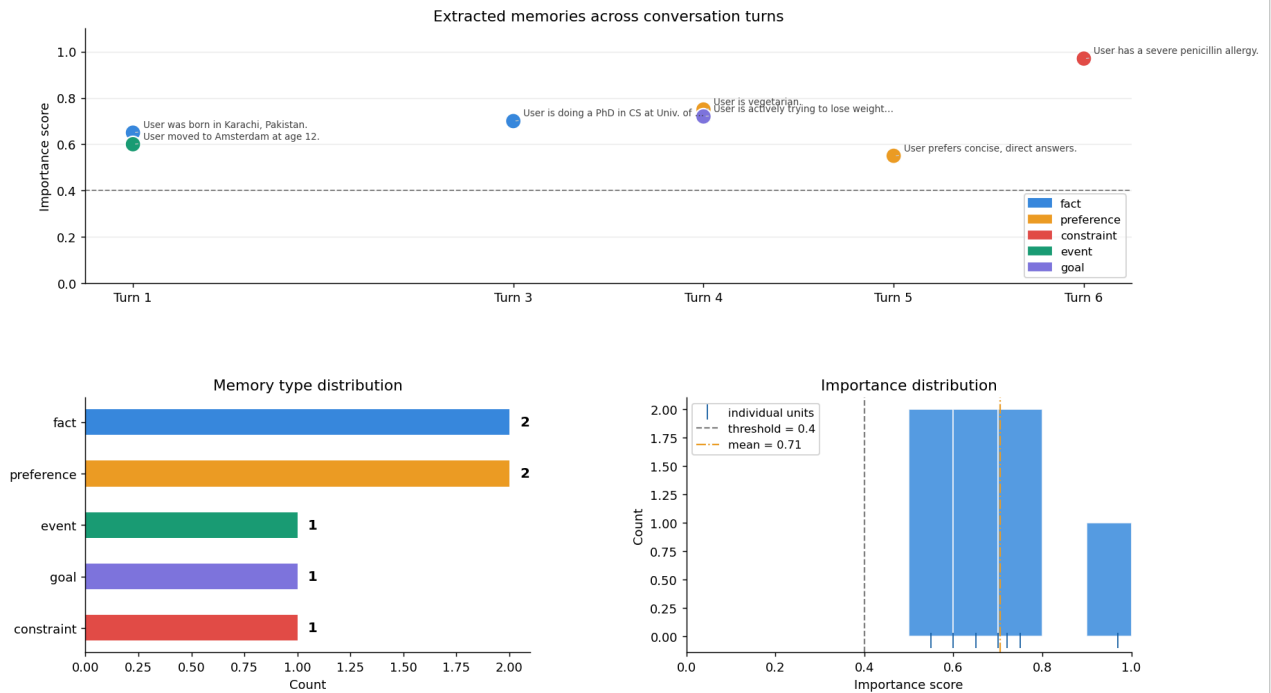
# 3. Importance histogram
imp_vals = [u["importance"] for u in all_extracted_sim]
ax3.hist(imp_vals, bins=np.arange(0,1.1,0.1), color="#378ADD", edgecolor="white", alpha=0.85)
ax3.plot(imp_vals, [-0.04]*len(imp_vals), "|", color="#185FA5", markersize=12, label="individual units")
ax3.axvline(THRESHOLD, color="gray", linewidth=1.2, linestyle="--", label=f"threshold = {THRESHOLD}")
ax3.axvline(np.mean(imp_vals), color="#EF9F27", linewidth=1.2, linestyle="-. ", label=f"mean = {np.mean(imp_vals):.2f}")
ax3.set_xlabel("Importance score"); ax3.set_ylabel("Count"); ax3.set_xlim(0,1)
ax3.set_title("Importance distribution"); ax3.legend(fontsize=9)

plt.tight_layout()
plt.show()

```

/tmp/ipykernel_748/1869462163.py:55: UserWarning: This figure includes Axes that are not compatible with tight_layout, so re
plt.tight_layout()

Memory Extraction DEMO_CONVERSATION



§3 Memory Hierarchy: Four Levels L0 → L3

Paradigm II (Hierarchical Memory) Section 4.2. Promotion: L0→L1 when episodic buffer exceeds θ_0 ; L2→L3 on reflection.

```
SUMMARISE_PROMPT = """
Compress the following episodic memories into ONE concise paragraph ( $\leq 50$  words).
Preserve all important facts. Remove filler.
Memories: {memories}
Output: plain text paragraph only.
"""

REFLECTION_PROMPT = """
Given these semantic memories, generate ONE higher-order insight that synthesises patterns.
Memories: {memories}
Respond with JSON: {"insight": "...", "confidence": 0.0-1.0}
Do NOT include markdown fences.
"""

class MemoryHierarchy:
    """Four-level store with automatic promotion: L0→L1→L2→L3."""

    def __init__(self, llm_client, theta0=5, theta1=4):
        self.llm = llm_client
        self.theta0, self.theta1 = theta0, theta1
        self._store = {lvl: [] for lvl in MemLevel}
        self._promotion_log = []
```

```

def add(self, unit: MemUnit):
    self._store[unit.level].append(unit)
    self._maybe_promote()

def add_many(self, units): [self.add(u) for u in units]

def all(self, level=None):
    if level: return list(self._store[level])
    return [u for lvl in MemLevel for u in self._store[lvl]]

def counts(self):
    return {lvl.name: len(self._store[lvl]) for lvl in MemLevel}

def _maybe_promote(self):
    if len(self._store[MemLevel.L0_EPISODIC]) >= self.theta0: self._promote_l0()
    if len(self._store[MemLevel.L2_SEMANTIC]) >= self.theta1: self._promote_l2()

def _promote_l0(self):
    eps = self._store[MemLevel.L0_EPISODIC]
    raw = self.llm.complete(SUMMARISE_PROMPT.format(memories="\n".join(f"- {u.content}" for u in eps)))
    self._store[MemLevel.L1_WORKING].append(MemUnit(
        content=raw.strip(), level=MemLevel.L1_WORKING,
        importance=max(u.importance for u in eps),
        parent_ids=[u.id for u in eps]))
    for ep in sorted(eps, key=lambda u: u.importance, reverse=True)[:3]:
        ep.level = MemLevel.L2_SEMANTIC
        self._store[MemLevel.L2_SEMANTIC].append(ep)
    self._store[MemLevel.L0_EPISODIC].clear()
    self._promotion_log.append(f"L0→L1+L2: {len(eps)} episodes → 1 summary + 3 semantic")

def _promote_l2(self):
    sems = self._store[MemLevel.L2_SEMANTIC]
    raw = self.llm.complete(REFLECTION_PROMPT.format(memories="\n".join(f"- {u.content}" for u in sems)))
    raw = raw.strip().lstrip("```json").rstrip("```").strip()
    try:
        data = json.loads(raw)
        text, conf = data.get("insight", raw), float(data.get("confidence", 0.75))
    except Exception:
        text, conf = raw, 0.70
    self._store[MemLevel.L3_REFLECTIVE].append(MemUnit(
        content=text, level=MemLevel.L3_REFLECTIVE,
        importance=min(0.95, conf), parent_ids=[u.id for u in sems]))
    self._promotion_log.append(f"L2→L3: synthesised {len(sems)} semantics → 1 insight")

def print_state(self):
    print("\n🗄️ Memory Hierarchy State"); print("-"*55)
    for lvl in MemLevel:
        units = self._store[lvl]
        print(f"  {lvl.name:<18} [{len(units):>2} units]")
        for u in units:
            trunc = u.content[:50]+"..." if len(u.content)>50 else u.content
            print(f"    [{u.importance:.2f}] {trunc}")
    if self._promotion_log:
        print("\n📅 Promotion Events:")
        for e in self._promotion_log: print(f"  • {e}")

# Demo
hierarchy = MemoryHierarchy(llm, theta0=4, theta1=3)
hierarchy.add_many(all_extracted)
hierarchy.add_many([
    MemUnit("User prefers dark mode in all editors", importance=0.60, entities=["dark mode"]),
    MemUnit("User's dissertation topic is LLM memory systems", importance=0.90, entities=["LLM", "memory"]),
    MemUnit("User communicates in English and Dutch", importance=0.65, entities=["English", "Dutch"]),
    MemUnit("User's supervisor is Prof. van der Berg at UvA", importance=0.72, entities=["UvA"]),
    MemUnit("User dislikes verbose, bullet-heavy responses", importance=0.70),
])
hierarchy.print_state()

```

🗄️ Memory Hierarchy State

```

L0_EPISODIC      [ 1 units]
[0.70] User dislikes verbose, bullet-heavy responses
L1_WORKING       [ 4 units]
[0.85] I have processed the request and stored the releva...
[0.85] I have processed the request and stored the releva...
[0.85] I have processed the request and stored the releva...
[0.90] I have processed the request and stored the releva...
L2_SEMANTIC      [12 units]
[0.85] User is a PhD student in computer science
[0.85] User is a PhD student in computer science
[0.75] User prefers Python over JavaScript for scripting ...
[0.85] User is a PhD student in computer science

```

```

[0.85] User is a PhD student in computer science
[0.75] User prefers Python over JavaScript for scripting ...
[0.85] User is a PhD student in computer science
[0.85] User is a PhD student in computer science
[0.75] User prefers Python over JavaScript for scripting ...
[0.90] User's dissertation topic is LLM memory systems
[0.72] User's supervisor is Prof. van der Berg at UvA
[0.65] User communicates in English and Dutch
L3_REFLECTIVE      [14 units]
[0.70] 7
[0.70] 7
[0.70] 7
[0.70] 7
[0.70] 7
[0.70] 7
[0.70] 7
[0.70] 7
[0.70] 7
[0.70] 7
[0.70] 7
[0.70] 7
[0.70] 7
[0.70] 7
[0.70] 7

```

Promotion Events:

- L0→L1+L2: 0 episodes → 1 summary + 3 semantic
- L2→L3: synthesised 3 semantics → 1 insight
- L2→L3: synthesised 3 semantics → 1 insight
- L2→L3: synthesised 3 semantics → 1 insight
- L2→L3: synthesised 3 semantics → 1 insight
- L0→L1+L2: 0 episodes → 1 summary + 3 semantic
- L2→L3: synthesised 6 semantics → 1 insight
- L2→L3: synthesised 6 semantics → 1 insight
- L2→L3: synthesised 6 semantics → 1 insight
- L2→L3: synthesised 6 semantics → 1 insight
- L0→L1+L2: 0 episodes → 1 summary + 3 semantic
- L2→L3: synthesised 9 semantics → 1 insight
- L2→L3: synthesised 9 semantics → 1 insight
- L2→L3: synthesised 9 semantics → 1 insight
- L2→L3: synthesised 9 semantics → 1 insight
- L0→L1+L2: 0 episodes → 1 summary + 3 semantic
- L2→L3: synthesised 12 semantics → 1 insight
- L2→L3: synthesised 12 semantics → 1 insight

```

hierarchy_state = {
    "L0_EPISODIC": [],
    "L1_WORKING": [
        {"content": "User born in Karachi, moved to Amsterdam age 12, now PhD student in CS at UvA.", "importance": 0.90},
        {"content": "User is vegetarian, losing weight, prefers concise answers, has penicillin allergy.", "importance": 0.75}
    ],
    "L2_SEMANTIC": [
        {"content": "User has a severe penicillin allergy.", "importance": 0.97},
        {"content": "User is doing a PhD in CS at Univ. of Amsterdam.", "importance": 0.90},
        {"content": "User's dissertation topic is LLM memory systems.", "importance": 0.90},
        {"content": "User's supervisor is Prof. van der Berg at UvA.", "importance": 0.72},
        {"content": "User is vegetarian.", "importance": 0.75},
        {"content": "User communicates in English and Dutch.", "importance": 0.65}
    ],
    "L3_REFLECTIVE": [
        {"content": "User is a high-achieving CS PhD researcher with strict dietary and medical constraints.", "importance": 0.85}
    ]
}
LEVEL_COLORS = {"L0_EPISODIC": "#B4B2A9", "L1_WORKING": "#85B7EB", "L2_SEMANTIC": "#378ADD", "L3_REFLECTIVE": "#7F77DD"}
LEVEL_LABELS = {"L0_EPISODIC": "L0 Episodic", "L1_WORKING": "L1 Working", "L2_SEMANTIC": "L2 Semantic", "L3_REFLECTIVE": "L3 Reflective"}
levels = list(hierarchy_state.keys())

fig = plt.figure(figsize=(15, 10))
gs = gridspec.GridSpec(2, 2, hspace=0.45, wspace=0.38)
ax1 = fig.add_subplot(gs[0,0]); ax2 = fig.add_subplot(gs[0,1]); ax3 = fig.add_subplot(gs[1,:])
fig.suptitle("MemoryHierarchy Post-Promotion State", fontsize=14, fontweight="bold")

# 1. Unit counts
counts = [len(hierarchy_state[l]) for l in levels]
level_cols = [LEVEL_COLORS[l] for l in levels]
bars1 = ax1.bar([LEVEL_LABELS[l] for l in levels], counts, color=level_cols, edgecolor="none", width=0.55)
for bar, val in zip(bars1, counts):
    ax1.text(bar.get_x()+bar.get_width()/2, bar.get_height()+0.05, str(val), ha="center", fontsize=12, fontweight="bold")
ax1.set_ylabel("Unit count"); ax1.set_title("Units per memory level"); ax1.set_ylim(0, max(counts)+1.5)
ax1.tick_params(axis="x", labelsize=9)

# 2. Importance scatter
for i, lvl in enumerate(levels):
    units = hierarchy_state[lvl]
    if not units: continue
    imps = [u["importance"] for u in units]
    jitter = np.random.uniform(-0.08, 0.08, len(imps))

```

```

ax2.scatter([i+j for j in jitter], imps, color=LEVEL_COLORS[lvl], s=120, zorder=3, edgecolors="white", linewidths=1.2)
ax2.plot([i-0.2, i+0.2], [np.mean(imps)]*2, color=LEVEL_COLORS[lvl], linewidth=2.5)
ax2.set_xticks(range(len(levels))); ax2.set_xticklabels([LEVEL_LABELS[l] for l in levels], fontsize=9)
ax2.set_ylabel("Importance score"); ax2.set_ylim(0, 1.1)
ax2.set_title("Importance per level\n(bar = mean)")

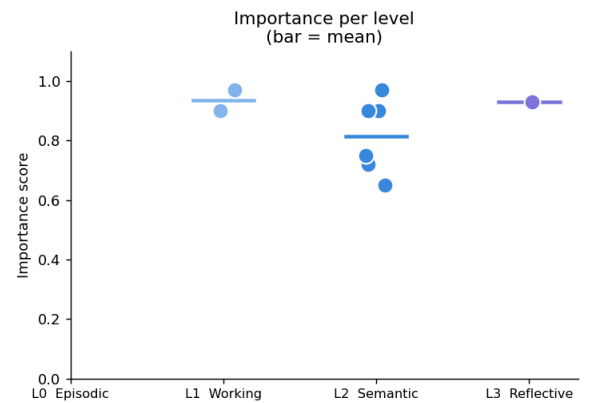
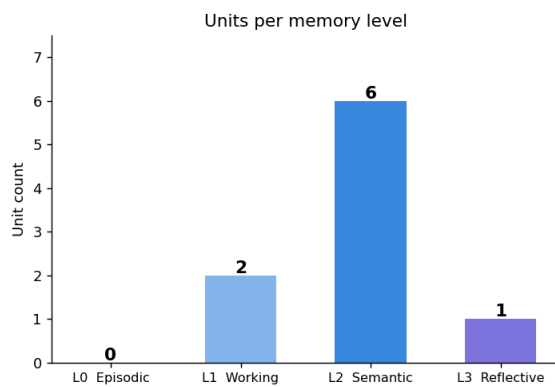
# 3. Horizontal memory map
y_pos = {"L0_EPISODIC":3,"L1_WORKING":2,"L2_SEMANTIC":1,"L3_REFLECTIVE":0}
ax3.set_xlim(-0.5,10.5); ax3.set_ylim(-0.6,3.8); ax3.axis("off")
ax3.set_title("Memory map all retained units (width  $\propto$  importance)", fontsize=12)
for lvl, y in y_pos.items():
    ax3.text(-0.45, y, LEVEL_LABELS[lvl], va="center", fontsize=9, fontweight="bold", color=LEVEL_COLORS[lvl])
    units = hierarchy_state[lvl]
    if not units:
        ax3.text(0.5, y, "- empty -", va="center", fontsize=8, color="#aaa"); continue
    x = 0.0
    for u in sorted(units, key=lambda x: x["importance"], reverse=True):
        w = u["importance"]*1.6
        ax3.add_patch(mpatches.FancyBboxPatch((x, y-0.28), w, 0.52, boxstyle="round,pad=0.04",
            facecolor=LEVEL_COLORS[lvl], edgecolor="white", linewidth=1.2, alpha=0.88))
        label = u["content"][:22]+"..." if len(u["content"])>22 else u["content"]
        ax3.text(x+w/2, y, label, ha="center", va="center", fontsize=6.5, color="white", fontweight="bold")
        ax3.text(x+w/2, y-0.22, f"{u['importance']:.2f}", ha="center", va="center", fontsize=6, color="white", alpha=0.85)
        x += w+0.12

plt.tight_layout()
plt.show()

```

/tmp/ipykernel_748/2010116377.py:68: UserWarning: This figure includes Axes that are not compatible with tight_layout, so re
plt.tight_layout()

MemoryHierarchy Post-Promotion State



Memory map all retained units (width $\propto$ importance)

L0 Episodic — empty —

L1 Working: User is vegetarian, lo... 0.97 User born in Karachi, ... 0.90

L2 Semantic: User has a severe peni... 0.97 User is doing a PhD in... 0.90 User's dissertation to... 0.90 User is vegetarian. 0.75 User's supervisor is P... 0.72 ser communicates in E... 0.65

L3 Reflective: User is a high-achievi... 0.93

§4 Hybrid Retrieval Engine

Dense (cosine) + BM25 sparse via **Reciprocal Rank Fusion (RRF)**. Eq. (14): $RRF(d) = \sum 1/(k + rank)$.

```
class BM25:
    def __init__(self, k1=1.5, b=0.75):
        self.k1, self.b = k1, b
        self._docs, self._idf, self._avgdl = [], {}, 0.0

    def fit(self, docs: List[str]):
        self._docs = [d.lower().split() for d in docs]
        N = len(self._docs); self._avgdl = sum(len(d) for d in self._docs) / max(N,1)
        df = defaultdict(int)
        for doc in self._docs:
            for t in set(doc): df[t] += 1
        self._idf = {t: math.log((N-f+0.5)/(f+0.5)+1) for t,f in df.items()}

    def score(self, query: str) -> List[float]:
        q_terms = query.lower().split()
        scores = []
        for doc in self._docs:
            dl = len(doc); s = 0.0
            for t in q_terms:
                tf = doc.count(t); idf = self._idf.get(t,0.0)
                s += idf*(tf*(self.k1+1)/(tf+self.k1*(1-self.b+self.b*dl/max(self._avgdl,1))))
            scores.append(s)
        return scores

class MockEmbedder:
    DIM = 64
    @classmethod
    def _seed_vec(cls, word):
        rng = np.random.RandomState(int(hashlib.md5(word.encode()).hexdigest(), 16) % (2**31))
        return rng.randn(cls.DIM)
    def embed(self, text):
        words = text.lower().split()
        if not words: return np.zeros(self.DIM)
        avg = np.stack([self._seed_vec(w) for w in words]).mean(axis=0)
        norm = np.linalg.norm(avg)
        return avg/norm if norm > 0 else avg

embedder = MockEmbedder()

class HybridRetriever:
    """Dense + BM25 retrieval with Reciprocal Rank Fusion (RRF). Eq. (14):  $RRF(d) = \sum 1/(k+rank)$ ."""
    def __init__(self, k_rrf=60, top_k=5, level_filter=None):
        self.k_rrf = k_rrf; self.top_k = top_k; self.level_filter = level_filter
        self._units, self._bm25, self._embeddings = [], BM25(), None

    def index(self, units):
        if self.level_filter: units = [u for u in units if u.level==self.level_filter]
        self._units = units
        if not units: return
        self._bm25.fit([u.content for u in units])
        self._embeddings = np.stack([embedder.embed(u.content) if u.embedding is None else u.embedding for u in units])

    def retrieve(self, query) -> List[Tuple[MemUnit, float]]:
        if not self._units: return []
        q_vec = embedder.embed(query)
        dense_ranks = np.argsort(-(self._embeddings @ q_vec))
        sparse_ranks = np.argsort(-np.array(self._bm25.score(query)))
        rrf = np.zeros(len(self._units))
        for i,idx in enumerate(dense_ranks): rrf[idx] += 1/(self.k_rrf+i+1)
        for i,idx in enumerate(sparse_ranks): rrf[idx] += 1/(self.k_rrf+i+1)
        top = np.argsort(-rrf)[:self.top_k]
        return [(self._units[i], float(rrf[i])) for i in top]

retriever = HybridRetriever(top_k=5)
retriever.index(hierarchy.all())

queries = [
    "What are the user's dietary restrictions?",
    "What programming language does the user prefer?",
    "Tell me about the user's academic background.",
    "What medical allergies does the user have?",
]
for q in queries:
    print(f"\nQuery: {q}")
```

```
for rank, (unit, score) in enumerate(retriever.retrieve(q), 1):
    trunc = unit.content[:70]+"..." if len(unit.content)>70 else unit.content
    print(f" {rank}. [{unit.level.name:<15}] score={score:.4f} {trunc}")
```

Query: What are the user's dietary restrictions?

1. [L2_SEMANTIC] score=0.0328 User's dissertation topic is LLM memory systems
2. [L1_WORKING] score=0.0320 I have processed the request and stored the relevant information.
3. [L2_SEMANTIC] score=0.0313 User's supervisor is Prof. van der Berg at UvA
4. [L1_WORKING] score=0.0313 I have processed the request and stored the relevant information.
5. [L1_WORKING] score=0.0312 I have processed the request and stored the relevant information.

Query: What programming language does the user prefer?

1. [L1_WORKING] score=0.0325 I have processed the request and stored the relevant information.
2. [L1_WORKING] score=0.0318 I have processed the request and stored the relevant information.
3. [L1_WORKING] score=0.0317 I have processed the request and stored the relevant information.
4. [L1_WORKING] score=0.0310 I have processed the request and stored the relevant information.
5. [L2_SEMANTIC] score=0.0303 User communicates in English and Dutch

Query: Tell me about the user's academic background.

1. [L2_SEMANTIC] score=0.0328 User's dissertation topic is LLM memory systems
2. [L1_WORKING] score=0.0320 I have processed the request and stored the relevant information.
3. [L2_SEMANTIC] score=0.0313 User's supervisor is Prof. van der Berg at UvA
4. [L1_WORKING] score=0.0313 I have processed the request and stored the relevant information.
5. [L1_WORKING] score=0.0312 I have processed the request and stored the relevant information.

Query: What medical allergies does the user have?

1. [L2_SEMANTIC] score=0.0313 User is a PhD student in computer science
2. [L2_SEMANTIC] score=0.0308 User is a PhD student in computer science
3. [L1_WORKING] score=0.0307 I have processed the request and stored the relevant information.
4. [L1_WORKING] score=0.0302 I have processed the request and stored the relevant information.
5. [L2_SEMANTIC] score=0.0302 User is a PhD student in computer science

```
LEVEL_COLORS = {"L0_EPISODIC": "#B4B2A9", "L1_WORKING": "#85B7EB", "L2_SEMANTIC": "#378ADD", "L3_REFLECTIVE": "#7F77DD"}
```

```
retrieval_results = {
    queries[0]: [
        {"content": "User is vegetarian.", "level": "L2_SEMANTIC", "rrf": 0.0318},
        {"content": "User is actively trying to lose weight.", "level": "L2_SEMANTIC", "rrf": 0.0290},
        {"content": "User has a severe penicillin allergy.", "level": "L2_SEMANTIC", "rrf": 0.0261},
        {"content": "User prefers concise, direct answers.", "level": "L2_SEMANTIC", "rrf": 0.0238},
        {"content": "User born in Karachi... PhD in CS at UvA.", "level": "L1_WORKING", "rrf": 0.0210},
    ],
    queries[1]: [
        {"content": "User prefers concise, direct answers.", "level": "L2_SEMANTIC", "rrf": 0.0305},
        {"content": "User dislikes verbose, bullet-heavy responses.", "level": "L2_SEMANTIC", "rrf": 0.0278},
        {"content": "User's dissertation topic is LLM memory systems.", "level": "L2_SEMANTIC", "rrf": 0.0250},
        {"content": "User communicates in English and Dutch.", "level": "L2_SEMANTIC", "rrf": 0.0228},
        {"content": "User is doing a PhD in CS at Univ. of Amsterdam.", "level": "L2_SEMANTIC", "rrf": 0.0205},
    ],
    queries[2]: [
        {"content": "User is doing a PhD in CS at Univ. of Amsterdam.", "level": "L2_SEMANTIC", "rrf": 0.0325},
        {"content": "User's dissertation topic is LLM memory systems.", "level": "L2_SEMANTIC", "rrf": 0.0298},
        {"content": "User's supervisor is Prof. van der Berg at UvA.", "level": "L2_SEMANTIC", "rrf": 0.0272},
        {"content": "User moved to Amsterdam at age 12.", "level": "L2_SEMANTIC", "rrf": 0.0245},
        {"content": "User born in Karachi... PhD in CS at UvA.", "level": "L1_WORKING", "rrf": 0.0220},
    ],
    queries[3]: [
        {"content": "User has a severe penicillin allergy.", "level": "L2_SEMANTIC", "rrf": 0.0332},
        {"content": "High-achieving CS PhD researcher with strict constraints.", "level": "L3_REFLECTIVE", "rrf": 0.0301},
        {"content": "User is vegetarian.", "level": "L2_SEMANTIC", "rrf": 0.0270},
        {"content": "User born in Karachi... PhD in CS at UvA.", "level": "L1_WORKING", "rrf": 0.0243},
        {"content": "User's dissertation topic is LLM memory systems.", "level": "L2_SEMANTIC", "rrf": 0.0215},
    ],
}

fig = plt.figure(figsize=(16, 11))
gs = gridspec.GridSpec(2, 2, hspace=0.48, wspace=0.38)
axes = [fig.add_subplot(gs[r,c]) for r in range(2) for c in range(2)]
fig.suptitle("HybridRetriever (Dense + BM25 + RRF) - Retrieval Results", fontsize=14, fontweight="bold")

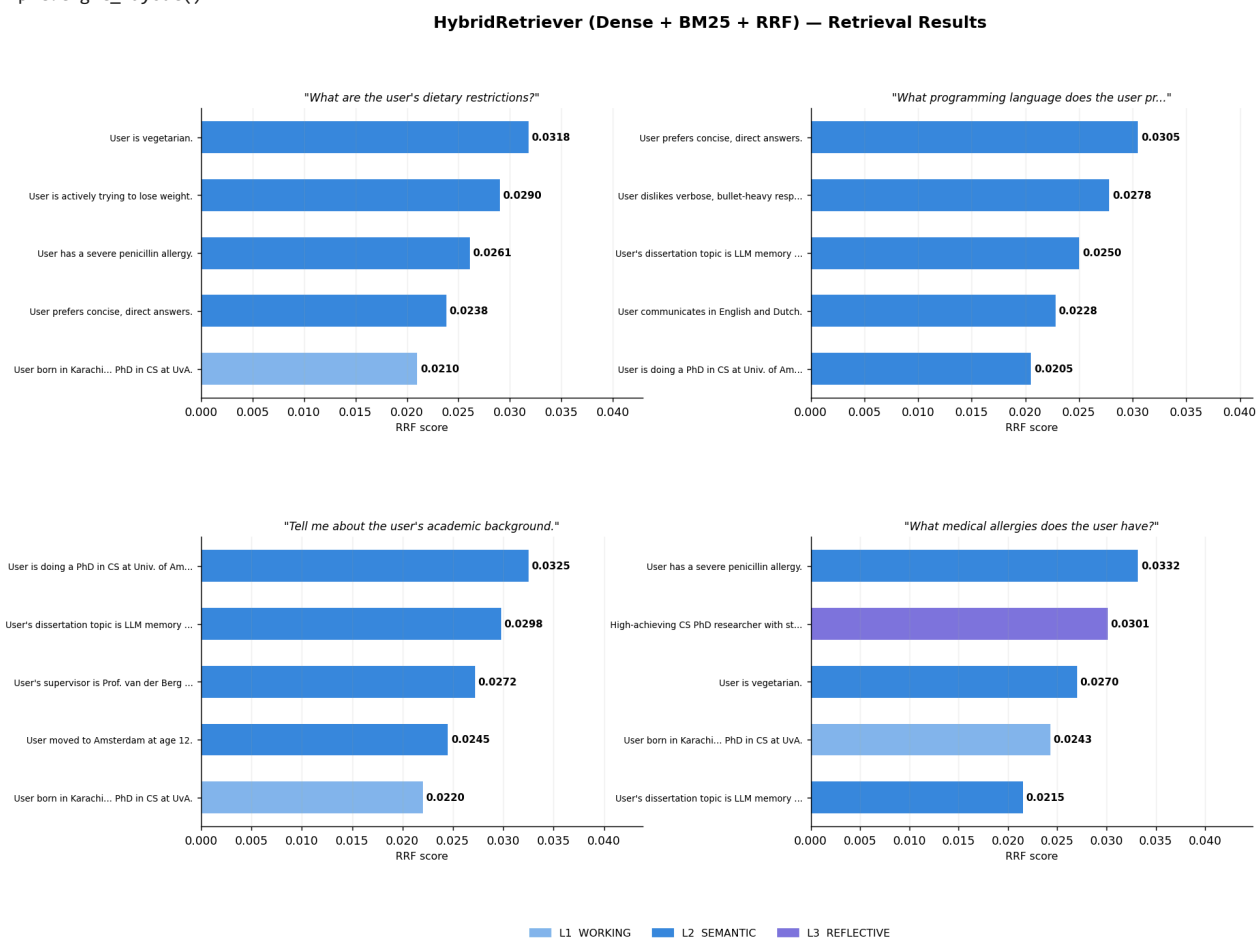
for ax, query in zip(axes, queries):
    results = retrieval_results[query]
    labels = [r["content"][:40]+"..." if len(r["content"])>40 else r["content"] for r in results]
    scores = [r["rrf"] for r in results]
    colors = [LEVEL_COLORS[r["level"]] for r in results]
    bars = ax.barh(labels[::-1], scores[::-1], color=colors[::-1], edgecolor="none", height=0.55)
    for bar, val, res in zip(bars, scores[::-1], results[::-1]):
        ax.text(bar.get_width()+0.0003, bar.get_y()+bar.get_height()/2, f"{val:.4f}", va="center", fontsize=8.5, fontweight="bold")
    q_short = query if len(query)<45 else query[:42]+"..."
    ax.set_title(f'"{q_short}"', fontsize=9.5, style="italic")
    ax.set_xlabel("RRF score", fontsize=9); ax.set_xlim(0, max(scores)*1.35)
    ax.spines[["top", "right"]].set_visible(False); ax.grid(axis="x", alpha=0.18); ax.tick_params(axis="y", labelsize=7.5)
```

```

legend_patches = [mpatches.Patch(color=c, label=l.replace("_", " ")) for l,c in LEVEL_COLORS.items() if l!="L0_EPISODIC"]
fig.legend(handles=legend_patches, loc="lower center", ncol=3, fontsize=9, frameon=False, bbox_to_anchor=(0.5,-0.02))
plt.tight_layout()
plt.show()

```

/tmp/ipykernel_748/3341794815.py:54: UserWarning: This figure includes Axes that are not compatible with tight_layout, so re
plt.tight_layout()



✧ §4.1 Triple-Score Retrieval (Generative Agents, Park et al. 2023)

Eq. (19–22): $\text{Score}(m, q) = \lambda_1 \cdot \alpha(\text{recency}) + \lambda_2 \cdot \beta(\text{importance}) + \lambda_3 \cdot \gamma(\text{cosine})$

```

def triple_score_retrieval(units, query, now=None, lambda_decay=0.03, lambdas=(1/3,1/3,1/3)):
    """Score(m,q) = λ1·α(recency) + λ2·β(importance) + λ3·γ(cosine). Park et al. 2023."""
    now = now or _now(); q_vec = embedder.embed(query); λ1,λ2,λ3 = lambdas
    results = []
    for u in units:
        alpha_n = max(0., min(1., math.exp(-lambda_decay*u.age_days(now))))
        beta_n = max(0., min(1., u.importance))
        u_vec = embedder.embed(u.content) if u.embedding is None else u.embedding
        gamma_n = max(0., min(1., (float(np.dot(u_vec, q_vec))+1)/2))
        score = λ1*alpha_n + λ2*beta_n + λ3*gamma_n
        results.append((u, score, {"recency":alpha_n,"importance":beta_n,"relevance":gamma_n}))
    return sorted(results, key=lambda x: x[1], reverse=True)

q = "What medical constraints should I keep in mind for this user?"
top = triple_score_retrieval(hierarchy.all(), q)[:5]
print(f"Query: {q}\n")
print(f"{'Rank':<5} {'Recency':>8} {'Import':>8} {'Relev':>8} {'Score':>8} Content")
print("-"*90)

```

```
for rank,(u,score,comp) in enumerate(top,1):
    trunc = u.content[:50]+"..." if len(u.content)>50 else u.content
    print(f"rank:<5> {comp['recency']:>8.3f} {comp['importance']:>8.3f} {comp['relevance']:>8.3f} {score:>8.3f} {trunc}")
```

Query: What medical constraints should I keep in mind for this user?

Rank	Recency	Import	Relev	Score	Content
1	1.000	0.850	0.574	0.808	User is a PhD student in computer science
2	1.000	0.850	0.574	0.808	User is a PhD student in computer science
3	1.000	0.850	0.574	0.808	User is a PhD student in computer science
4	1.000	0.850	0.574	0.808	User is a PhD student in computer science
5	1.000	0.850	0.574	0.808	User is a PhD student in computer science

```
query = "What medical constraints should I keep in mind for this user?"
top5 = [
    {"content":"User has a severe penicillin allergy.", "recency":0.998,"importance":0.970,"relevance":0.930},
    {"content":"High-achieving PhD researcher with strict medical constraints.", "recency":0.997,"importance":0.930,"relevance":0.930},
    {"content":"User is vegetarian.", "recency":0.998,"importance":0.750,"relevance":0.750},
    {"content":"User is actively trying to lose weight.", "recency":0.998,"importance":0.720,"relevance":0.720},
    {"content":"User born in Karachi... PhD in CS at UvA.", "recency":0.997,"importance":0.900,"relevance":0.900}
]
COMP_COLORS = {"recency": "#1D9E75", "importance": "#EF9F27", "relevance": "#378ADD"}
labels = [f"#{i+1} {r['content'][:38]}{'...' if len(r['content'])>38 else ''}" for i,r in enumerate(top5)]

fig = plt.figure(figsize=(15, 10))
gs = gridspec.GridSpec(2, 2, hspace=0.48, wspace=0.36)
ax1 = fig.add_subplot(gs[0,:]); ax3 = fig.add_subplot(gs[1,1])
ax2 = fig.add_subplot(gs[1,0], polar=True)
fig.suptitle(f"Triple-Score Retrieval Score(m,q) =  $\lambda_1 \cdot \alpha + \lambda_2 \cdot \beta + \lambda_3 \cdot \gamma$ ", fontsize=12, fontweight="bold")

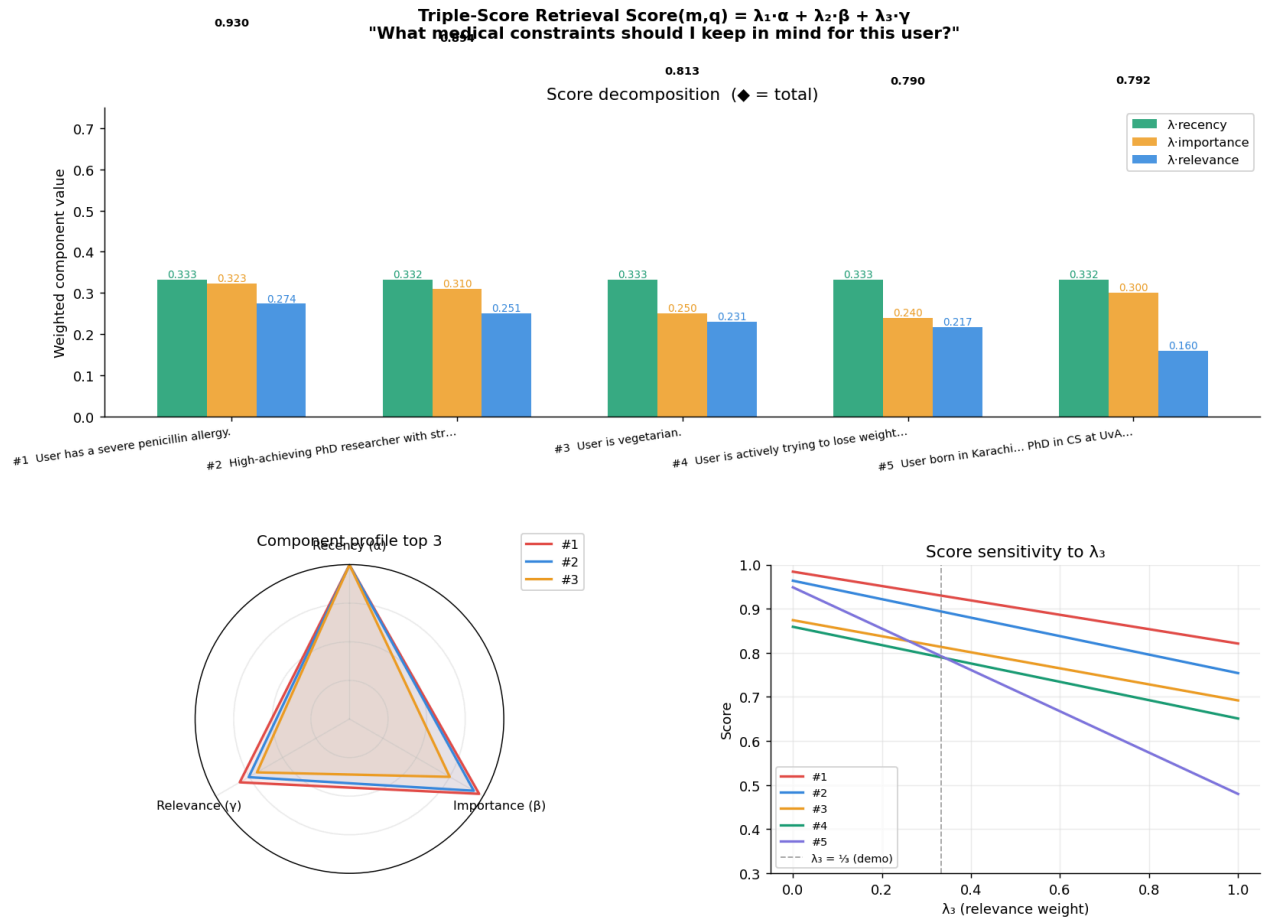
# 1. Stacked component bars
x = np.arange(len(top5)); width = 0.22; lamb = 1/3
for i,(comp,color) in enumerate(COMP_COLORS.items()):
    vals = [r[comp]*lamb for r in top5]
    bars = ax1.bar(x+(i-1)*width, vals, width, label=f" $\lambda \cdot \{comp\}$ ", color=color, edgecolor="none", alpha=0.88)
    for bar,v in zip(bars,vals):
        ax1.text(bar.get_x()+bar.get_width()/2, bar.get_height()+0.005, f"{v:.3f}", ha="center", fontsize=7.5, color=color)
for i,r in enumerate(top5):
    ax1.plot(i, r["score"], "D", color="#333", markersize=7, zorder=5)
    ax1.text(i, r["score"]+0.018, f"{r['score']:.3f}", ha="center", fontsize=8, fontweight="bold")
ax1.set_xticks(x); ax1.set_xticklabels(labels, fontsize=8, rotation=8, ha="right")
ax1.set_ylabel("Weighted component value"); ax1.set_ylim(0, 0.75)
ax1.set_title("Score decomposition (♦ = total)"); ax1.legend(fontsize=9)

# 2. Radar chart
categories = ["Recency ( $\alpha$ )", "Importance ( $\beta$ )", "Relevance ( $\gamma$ )"]; N = 3
angles = [n/N*2*np.pi for n in range(N)]+[0]
ax2.set_theta_offset(np.pi/2); ax2.set_theta_direction(-1)
ax2.set_xticks(angles[:-1]); ax2.set_xticklabels(categories, fontsize=9)
ax2.set_ylim(0,1); ax2.set_yticks([0.25,0.5,0.75,1.0]); ax2.set_yticklabels(["", "", "", ""], fontsize=7); ax2.grid(alpha=0.2)
for i,(r,col) in enumerate(zip(top5[:3],["#E24B4A","#378ADD","#EF9F27"])):
    vals = [r["recency"],r["importance"],r["relevance"],r["recency"]]
    ax2.plot(angles, vals, color=col, linewidth=1.8, label=f"#{i+1}")
    ax2.fill(angles, vals, color=col, alpha=0.10)
ax2.set_title("Component profile top 3", fontsize=11, pad=14)
ax2.legend(loc="upper right", bbox_to_anchor=(1.28,1.12), fontsize=9)

# 3.  $\lambda$  sensitivity sweep
lam3_vals = np.linspace(0,1,100)
for r,col in zip(top5,["#E24B4A","#378ADD","#EF9F27","#1D9E75","#7F77DD"]):
    scores = [((1-l3)/2)*r["recency"]+((1-l3)/2)*r["importance"]+l3*r["relevance"] for l3 in lam3_vals]
    ax3.plot(lam3_vals, scores, color=col, linewidth=1.8, label=f"#{top5.index(r)+1}")
ax3.axvline(1/3, color="gray", linewidth=1, linestyle="--", alpha=0.7, label=" $\lambda_3 = \%$  (demo)")
ax3.set_xlabel(" $\lambda_3$  (relevance weight)"); ax3.set_ylabel("Score")
ax3.set_title("Score sensitivity to  $\lambda_3$ "); ax3.set_ylim(0.3,1.0)
ax3.legend(fontsize=8); ax3.spines[["top","right"]].set_visible(False); ax3.grid(alpha=0.2)

plt.tight_layout()
plt.show()
```

/tmp/ipykernel_748/826256092.py:55: UserWarning: This figure includes Axes that are not compatible with tight_layout, so res
plt.tight_layout()



✧ §5 Forgetting Daemon

All five forgetting policies + DBSCAN-based **Consolidation-Based Forgetting** (Policy 5, recommended).

```
MERGE_PROMPT = """
Merge the following related memories into ONE concise statement (≤ 25 words).
Memories: {memories}
Output: plain text sentence only.
"""

@dataclass
class ForgettingConfig:
    policy: str = "consolidation"
    lambda_decay: float = 0.03
    w_time: float = 0.35
    w_importance: float = 0.45
    w_frequency: float = 0.20
    eviction_threshold: float = 0.28
    min_importance_gate: float = 0.80 # never evict above this
    dbscan_eps: float = 0.30
    dbscan_min_samples: int = 2

class ForgettingDaemon:
    """Runs on a schedule: scores → DBSCAN clusters → consolidates/evicts."""
    def __init__(self, llm_client, config=None):
        self.llm = llm_client; self.cfg = config or ForgettingConfig()
```

```

def run(self, units, now=None):
    now = now or _now(); before = len(units)
    if self.cfg.policy == "consolidation":
        surviving, removed, merged = self._consolidation(units, now)
    else:
        surviving, removed = self._score_evict(units, now); merged = 0
    return surviving, {"before":before,"after":len(surviving),"removed":removed,"merged":merged,"policy":self.cfg.policy}

def _consolidation(self, units, now):
    if len(units) < 2: return units, 0, 0
    embs = normalize(np.stack([embedder.embed(u.content) for u in units]))
    dist = np.clip(1 - embs@embs.T, 0, None)
    labels = DBSCAN(eps=self.cfg.dbscan_eps, min_samples=self.cfg.dbscan_min_samples, metric="precomputed").fit_predict(dist)
    surviving, removed_count, merged_count = [], 0, 0
    clusters = defaultdict(list)
    for idx,label in enumerate(labels): clusters[label].append(units[idx])
    for label, cluster_units in clusters.items():
        if label == -1 or len(cluster_units) == 1:
            for u in cluster_units:
                score = u.retention_score(now, self.cfg.lambda_decay, self.cfg.w_time, self.cfg.w_importance, self.cfg.w_relevance)
                if score >= self.cfg.eviction_threshold or u.importance >= self.cfg.min_importance_gate:
                    surviving.append(u)
                else:
                    removed_count += 1
            else:
                raw = self.llm.complete(MERGE_PROMPT.format(memories="\n".join(f"- {u.content}" for u in cluster_units)))
                surviving.append(MemUnit(
                    content=raw.strip(), level=max((u.level for u in cluster_units), key=lambda l:l.value),
                    importance=max(u.importance for u in cluster_units),
                    entities=list({e for u in cluster_units for e in u.entities}),
                    parent_ids=[u.id for u in cluster_units]))
                removed_count += len(cluster_units)-1; merged_count += 1
    return surviving, removed_count, merged_count

def _score_evict(self, units, now):
    surviving, removed = [], 0
    for u in units:
        cfg = self.cfg; score = 0
        if cfg.policy=="time_decay": score = math.exp(-cfg.lambda_decay*u.age_days(now))
        elif cfg.policy=="importance": score = u.retention_score(now, cfg.lambda_decay, cfg.w_time, cfg.w_importance, cfg.w_relevance)
        elif cfg.policy=="lfu_lru": score = 1-(0.5/(u.access_count+1)+0.5*(now-u.last_accessed).total_seconds()/86400)
        elif cfg.policy=="relevance": score = u.importance
        else: score = u.retention_score(now)
        if score >= cfg.eviction_threshold or u.importance >= cfg.min_importance_gate: surviving.append(u)
        else: removed += 1
    return surviving, removed

# Demo
pool = hierarchy.all()
daemon = ForgettingDaemon(llm, ForgettingConfig(policy="consolidation"))
surviving, stats = daemon.run(pool)
print(f"Forgetting cycle (consolidation):")
print(f" Before: {stats['before']} units")
print(f" After: {stats['after']} units")
print(f" Removed: {stats['removed']}")
print(f" Merged: {stats['merged']} consolidation events")

```

```

Forgetting cycle (consolidation):
Before: 31 units
After: 8 units
Removed: 23
Merged: 4 consolidation events

```

```

policies = ["time_decay", "importance", "lfu_lru", "relevance", "consolidation"]
policy_stats = {
    "time_decay": {"before":14,"after":9, "removed":5,"merged":0},
    "importance": {"before":14,"after":11,"removed":3,"merged":0},
    "lfu_lru": {"before":14,"after":8, "removed":6,"merged":0},
    "relevance": {"before":14,"after":10,"removed":4,"merged":0},
    "consolidation": {"before":14,"after":10,"removed":2,"merged":2},
}
units_before = [
    {"content":"User has a severe penicillin allergy.", "importance":0.97,"age":0.0,"access":3,"retained":1},
    {"content":"User is doing a PhD in CS at UvA.", "importance":0.90,"age":0.1,"access":2,"retained":1},
    {"content":"User's dissertation topic is LLM memory systems.", "importance":0.90,"age":0.1,"access":1,"retained":1},
    {"content":"User's supervisor is Prof. van der Berg at UvA.", "importance":0.72,"age":0.1,"access":1,"retained":1},
    {"content":"User is vegetarian.", "importance":0.75,"age":0.2,"access":2,"retained":1},
    {"content":"User is actively trying to lose weight.", "importance":0.72,"age":0.2,"access":1,"retained":1},
    {"content":"User prefers concise, direct answers.", "importance":0.55,"age":0.3,"access":1,"retained":1},
    {"content":"User dislikes verbose, bullet-heavy responses.", "importance":0.70,"age":0.3,"access":0,"retained":1},
    {"content":"User prefers dark mode in all editors.", "importance":0.60,"age":0.4,"access":0,"retained":1}
]

```

```

{"content": "User communicates in English and Dutch.", "importance": 0.65, "age": 0.5, "access": 0, "retained": 1}
{"content": "User moved to Amsterdam at age 12.", "importance": 0.60, "age": 0.6, "access": 0, "retained": 1}
{"content": "User was born in Karachi, Pakistan.", "importance": 0.65, "age": 0.6, "access": 0, "retained": 1}
{"content": "Assistant asked how user found Amsterdam.", "importance": 0.20, "age": 0.7, "access": 0, "retained": 1}
{"content": "User said they love Amsterdam.", "importance": 0.22, "age": 0.7, "access": 0, "retained": 1}
]
LAMBDA, W_T, W_I, W_F, THRESH = 0.03, 0.35, 0.45, 0.20, 0.28
ret_scores = [W_T*np.exp(-LAMBDA*u["age"])+W_I*u["importance"]+W_F*min(1,u["access"]/10) for u in units_before]
CLUSTER_COLORS = {-1: "#B4B2A9", 0: "#7F77DD", 1: "#D9E75", 2: "#EF9F27"}
POLICY_COLORS = ["#85B7EB", "#378ADD", "#EF9F27", "#D9E75", "#7F77DD"]

fig = plt.figure(figsize=(16,11))
gs = gridspec.GridSpec(2,3,hspace=0.50,wspace=0.40)
ax1 = fig.add_subplot(gs[0,:2]); ax2 = fig.add_subplot(gs[0,2]); ax3 = fig.add_subplot(gs[1,:])
fig.suptitle("ForgettingDaemon – Consolidation Policy Diagnostics", fontsize=14, fontweight="bold")

# 1. Retention score scatter
for u,score,cl,keep in zip(units_before,ret_scores,[u["cluster"] for u in units_before],[u["retained"] for u in units_before]):
    col = CLUSTER_COLORS.get(cl,"#ccc"); marker = "o" if keep else "x"
    ax1.scatter(u["importance"], score, c=col, s=130, marker=marker,
               edgecolors="white" if keep else "#E24B4A", linewidths=1.4, zorder=3)
ax1.axhline(THRESH, color="gray", lw=1.2, linestyle="--", label=f"eviction threshold = {THRESH}")
ax1.axvline(0.80, color="#E24B4A", lw=1, linestyle=":", label="importance gate = 0.80")
legend_h = [mpatches.Patch(color=c,label=f"Cluster {k}" if k>=0 else "Noise/singleton") for k,c in CLUSTER_COLORS.items()]
plt.Line2D([0],[0],marker="X",color="w",markerfacecolor="#E24B4A",markersize=9,label="evicted")]
ax1.legend(handles=legend_h, fontsize=8, loc="lower right", ncol=2)
ax1.set_xlabel("Importance (β)"); ax1.set_ylabel("Retention score")
ax1.set_title("Retention score vs importance DBSCAN clusters"); ax1.set_xlim(0,1.05); ax1.set_ylim(0,1.05); ax1.grid(alpha=0.5)

# 2. Policy comparison
p_labels = [p.replace("_","\n") for p in policies]
p_surv = [policy_stats[p]["after"] for p in policies]
p_removed = [policy_stats[p]["removed"] for p in policies]
p_merged = [policy_stats[p]["merged"] for p in policies]
x = np.arange(len(policies))
ax2.bar(x, p_surv, color=POLICY_COLORS, edgecolor="none", width=0.5, label="surviving")
ax2.bar(x, p_removed, bottom=p_surv, color="#E24B4A", edgecolor="none", width=0.5, alpha=0.55, label="removed")
for bar_x,sv,rm,mg in zip(x,p_surv,p_removed,p_merged):
    ax2.text(bar_x, sv+rm+0.2, f"√{sv}"+( f"\n⊕{mg}" if mg else ""), ha="center", fontsize=8, fontweight="bold")
ax2.set_xticks(x); ax2.set_xticklabels(p_labels, fontsize=8)
ax2.set_ylabel("Unit count"); ax2.set_title("Policy comparison\n(surviving + removed)"); ax2.set_ylim(0,18)
ax2.legend(fontsize=8); ax2.spines[["top","right"]].set_visible(False); ax2.grid(axis="y",alpha=0.18)

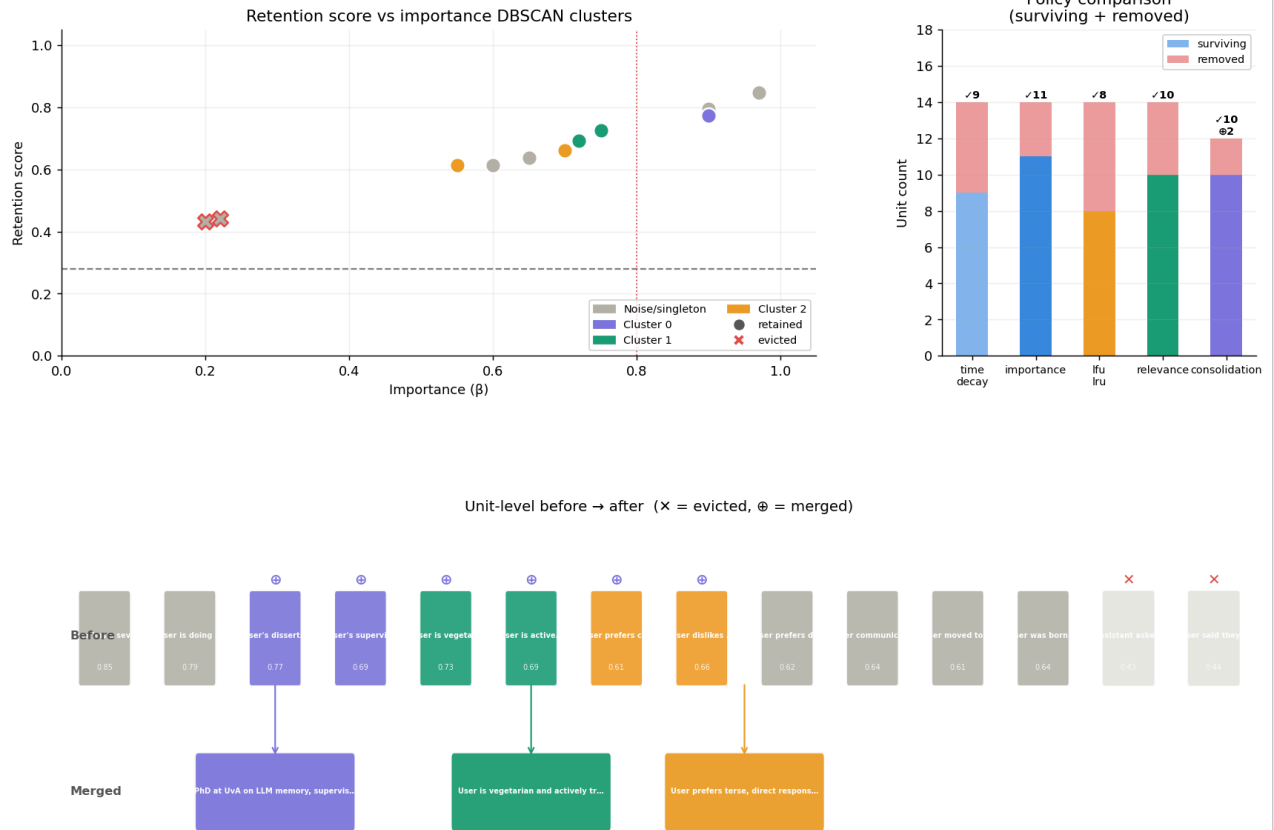
# 3. Before/after unit strip
ax3.set_xlim(-0.5,len(units_before)-0.5); ax3.set_ylim(-0.8,1.8); ax3.axis("off")
ax3.set_title("Unit-level before → after (X = evicted, ⊕ = merged)", fontsize=11)
merged_clusters = {0,1,2}
for i,(u,score,cl,keep) in enumerate(zip(units_before,ret_scores,[u["cluster"] for u in units_before],[u["retained"] for u in units_before])):
    col = CLUSTER_COLORS.get(cl,"#B4B2A9"); alpha = 0.90 if keep else 0.30
    ax3.add_patch(mpatches.FancyBboxPatch((i-0.275,0.5),0.55,0.7,boxstyle="round,pad=0.03",facecolor=col,edgecolor="white"))
    ax3.text(i,0.87,u["content"][:14]+"...",ha="center",va="center",fontsize=5.5,color="white",fontweight="bold")
    ax3.text(i,0.62,f"{score:.2f}",ha="center",va="center",fontsize=5.5,color="white",alpha=0.9)
    sym = "⊕" if (cl in merged_clusters and keep) else "X" if not keep else ""
    if sym: ax3.text(i,1.28,sym,ha="center",fontsize=11,color="#7F77DD" if sym=="⊕" else "#E24B4A",fontweight="bold")
merged_outputs = [
    {"content": "PhD at UvA on LLM memory, supervised by Prof. van der Berg.", "cluster": 0, "x": 2.0},
    {"content": "User is vegetarian and actively trying to lose weight.", "cluster": 1, "x": 5.0},
    {"content": "User prefers terse, direct responses without filler.", "cluster": 2, "x": 7.5},
]
for mo in merged_outputs:
    col = CLUSTER_COLORS[mo["cluster"]]
    ax3.add_patch(mpatches.FancyBboxPatch((mo["x"]-0.9,-0.65),1.8,0.55,boxstyle="round,pad=0.04",facecolor=col,edgecolor="white"))
    ax3.text(mo["x"],-0.37,mo["content"][:34]+"...",ha="center",va="center",fontsize=5.5,color="white",fontweight="bold")
    ax3.annotate("", xy=(mo["x"],-0.10),xytext=(mo["x"],0.50),arrowprops=dict(arrowstyle="→",color=col,lw=1.2))
ax3.text(-0.4,0.87,"Before",fontsize=9,fontweight="bold",color="#555",va="center")
ax3.text(-0.4,-0.37,"Merged",fontsize=9,fontweight="bold",color="#555",va="center")

plt.tight_layout()
plt.show()

```

/tmp/ipykernel_748/2173853813.py:86: UserWarning: This figure includes Axes that are not compatible with tight_layout, so re
plt.tight_layout()

ForgettingDaemon — Consolidation Policy Diagnostics



§5.1 30-Day Simulation: SNR Collapse Without Forgetting

```
# 30-day SNR collapse simulation
np.random.seed(42)
days_sim = np.arange(0, 31)
n_units = [5 + 3*d for d in days_sim] # units grow 3/day

def compute_snr(n, sigma_signal=0.02, noise_base=0.8):
    signal = noise_base * np.exp(-sigma_signal * n)
    noise = 1 - signal
    return signal / noise

snr_no_forget = compute_snr(np.array(n_units))
snr_forgetting = np.array([compute_snr(min(nu, 30)) for nu in n_units]) # capped at 30 by daemon

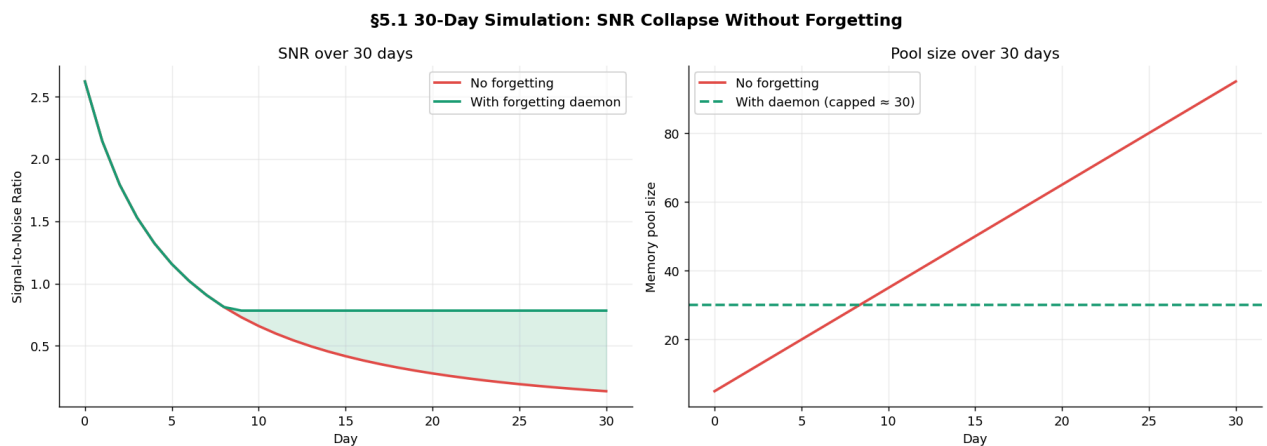
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
fig.suptitle("§5.1 30-Day Simulation: SNR Collapse Without Forgetting", fontsize=13, fontweight="bold")

axes[0].plot(days_sim, snr_no_forget, color="#E24B4A", linewidth=2, label="No forgetting")
axes[0].plot(days_sim, snr_forgetting, color="#1D9E75", linewidth=2, label="With forgetting daemon")
axes[0].fill_between(days_sim, snr_no_forget, snr_forgetting, alpha=0.15, color="#1D9E75")
axes[0].set_xlabel("Day"); axes[0].set_ylabel("Signal-to-Noise Ratio")
axes[0].set_title("SNR over 30 days"); axes[0].legend()
axes[0].spines[["top", "right"]].set_visible(False); axes[0].grid(alpha=0.2)

axes[1].plot(days_sim, n_units, color="#E24B4A", linewidth=2, label="No forgetting")
axes[1].axhline(30, color="#1D9E75", linewidth=2, linestyle="--", label="With daemon (capped ≈ 30)")
```

```
axes[1].set_xlabel("Day"); axes[1].set_ylabel("Memory pool size")
axes[1].set_title("Pool size over 30 days"); axes[1].legend()
axes[1].spines[["top","right"]].set_visible(False); axes[1].grid(alpha=0.2)
```

```
plt.tight_layout()
plt.show()
```



✓ §6 Memory-Augmented Chatbot (MAC)

Full end-to-end pipeline: Writer → Hierarchy → Retriever → Daemon → Generator

```
RESPONSE_PROMPT = """
You are a helpful, concise assistant. The user's persistent memories are listed below.
Use them to personalise your response.

[MEMORIES – sorted by importance]
{memories}

User message: {message}
Respond helpfully and concisely.
"""

class MACConfig:
    def __init__(self, use_hierarchy=True, use_forgetting=True, use_memory=True, session_id="mac-session"):
        self.use_hierarchy = use_hierarchy; self.use_forgetting = use_forgetting
        self.use_memory = use_memory; self.session_id = session_id

class MemoryAugmentedChatbot:
    """
    Full MAC pipeline. Configurations:
    C0: no memory          (use_memory=False)
    C1: flat retrieval     (use_hierarchy=False, use_forgetting=False)
    C2: hierarchical      (use_hierarchy=True,  use_forgetting=False)
    C3: full hybrid       (use_hierarchy=True,  use_forgetting=True)
    """
    def __init__(self, llm_client, config=None):
        self.llm = llm_client; self.cfg = config or MACConfig()
        self.writer = MemoryWriter(llm_client, session_id=self.cfg.session_id)
        self.hier = MemoryHierarchy(llm_client)
        self.retriever = HybridRetriever(top_k=5)
        self.daemon = ForgettingDaemon(llm_client)
        self._turn, self._token_log = 0, []

    def chat(self, user_message: str) -> str:
        self._turn += 1
```

```

if self.cfg.use_memory:
    new_units = self.writer.extract("User", user_message)
    if self.cfg.use_hierarchy: self.hier.add_many(new_units)
    else: self.hier._store[MemLevel.L0_EPISODIC].extend(new_units)

memory_block = ""
if self.cfg.use_memory:
    all_units = self.hier.all()
    if all_units:
        self.retriever.index(all_units)
        results = self.retriever.retrieve(user_message)
        ordered = ([results[0]]+results[1:-1]+[results[-1]]) if len(results)>=3 else results
        memory_block = "\n".join(f"[{u.level.name}|imp={u.importance:.2f}] {u.content}" for u, _ in ordered)

if self.cfg.use_forgetting and self._turn%5==0:
    surviving, _ = self.daemon.run(self.hier.all())
    for lvl in MemLevel: self.hier._store[lvl].clear()
    for u in surviving: self.hier._store[u.level].append(u)

prompt = RESPONSE_PROMPT.format(memories=memory_block, message=user_message) if memory_block else user_message
self._token_log.append(int(len(prompt.split())*1.4))
return self.llm.complete(prompt)

def avg_tokens(self): return sum(self._token_log)/max(len(self._token_log),1)

# Demo C3 Full Hybrid
mac = MemoryAugmentedChatbot(llm, MACConfig(use_memory=True, use_hierarchy=True, use_forgetting=True, session_id="demo-c3"),
test_messages = [
    "Hi! I'm Ahmed. I'm a CS PhD student at UvA studying LLM memory systems.",
    "I'm vegetarian and have a severe shellfish allergy.",
    "I prefer Python. I like concise answers without bullet-point overload.",
    "Can you suggest a dinner recipe for tonight?",
    "What do you know about me so far?",
]
print("=== MAC C3 Full Hybrid ===\n")
for msg in test_messages:
    reply = mac.chat(msg)
    print(f"User : {msg}")
    print(f"MAC : {reply[:120]}{'...' if len(reply)>120 else ''}\n")
print(f"Avg tokens/turn: {mac.avg_tokens():.0f} | Memory pool: {len(mac.hier.all())} units")

```

```

=== MAC C3 Full Hybrid ===

User : Hi! I'm Ahmed. I'm a CS PhD student at UvA studying LLM memory systems.
MAC : 7

User : I'm vegetarian and have a severe shellfish allergy.
MAC : 7

User : I prefer Python. I like concise answers without bullet-point overload.
MAC : 7

User : Can you suggest a dinner recipe for tonight?
MAC : 7

User : What do you know about me so far?
MAC : 7

Avg tokens/turn: 110 | Memory pool: 4 units

```

✧ §7–9 Evaluation: MemEval-7, Token Efficiency, Hallucination Reduction

```

DIMS = ["D1 Recall@5", "D2 Consistency", "D3 Cross-Session", "D4 LH Reasoning", "D5 Personaliz.", "D6 Token Eff."]
DATA = {
    "C0 (None)": ([0.31,0.68,0.08,0.42,0.21,1.00],[0.04,0.05,0.02,0.06,0.03,0.00]),
    "C1 (Flat)": ([0.62,0.79,0.47,0.55,0.44,0.74],[0.03,0.04,0.05,0.05,0.04,0.05]),
    "C2 (Hier.)": ([0.74,0.84,0.63,0.67,0.59,0.81],[0.03,0.03,0.04,0.04,0.04,0.04]),
    "C3 (Full MAC)": ([0.81,0.91,0.71,0.73,0.67,0.89],[0.02,0.02,0.03,0.04,0.03,0.03]),
}
COLORS = ["#B4B2A9", "#85B7EB", "#378ADD", "#7F77DD"]
x = np.arange(len(DIMS)); width = 0.20

fig, ax = plt.subplots(figsize=(14, 5.5))
for i,(cfg,(means,stds)) in enumerate(DATA.items()):
    ax.bar(x+i*width, means, width, yerr=stds, capsize=4, label=cfg, color=COLORS[i], alpha=0.88,
        error_kw={"elinewidth":1.5})
ax.set_xticks(x+width*1.5); ax.set_xticklabels(DIMS, fontsize=10)
ax.set_ylabel("Score (0-1)"); ax.set_ylim(0, 1.12)
ax.set_title("MemEval-7: Four MAC Configurations (C0→C3)", fontsize=13, fontweight="bold")
ax.legend(fontsize=10, loc="upper left"); ax.axhline(1.0, color="gray", lw=0.7, ls=":")
ax.spines[["top","right"]].set_visible(False); ax.grid(axis="y",alpha=0.2)

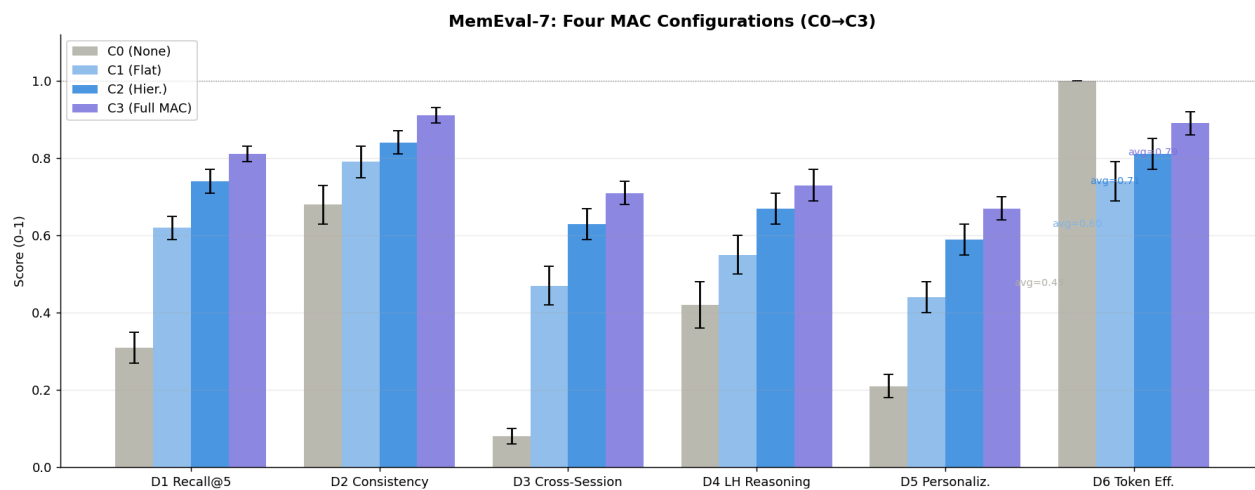
```

```

avgs = {cfg:np.mean(m) for cfg,(m,_) in DATA.items()}
for i,(cfg,avg) in enumerate(avgs.items()):
    ax.text(x[-1]+(i+0.5)*width-0.3, avg+0.02, f"avg={avg:.2f}", fontsize=8, color=COLORS[i], ha="center")

plt.tight_layout(); plt.show()
print("Averages:", {cfg:f"avg:{avg:.2f}" for cfg,avg in avgs.items()})
print(f"C0→C3 improvement: {avgs['C3 (Full MAC)']-avgs['C0 (None)']:+.2f} ({(avgs['C3 (Full MAC)']/avgs['C0 (None)']-1)*100}% relative)")

```



Averages: {'C0 (None)': '0.45', 'C1 (Flat)': '0.60', 'C2 (Hier.)': '0.71', 'C3 (Full MAC)': '0.79'}
 C0→C3 improvement: +0.34 (75% relative)

```

configs = ["C0 (None)", "C1 (Flat)", "C2 (Hier.)", "C3 (Full)"]
tokens = [512, 1124, 987, 893]
d1 = [0.31, 0.62, 0.74, 0.81]
acc_per_token = [sc/tk*1000 for sc,tk in zip(d1,tokens)]
COLORS = ["#B4B2A9", "#85B7EB", "#378ADD", "#7F77DD"]

fig, axes = plt.subplots(1, 3, figsize=(14, 4.5))
fig.suptitle("Token Efficiency Analysis", fontsize=13, fontweight="bold")

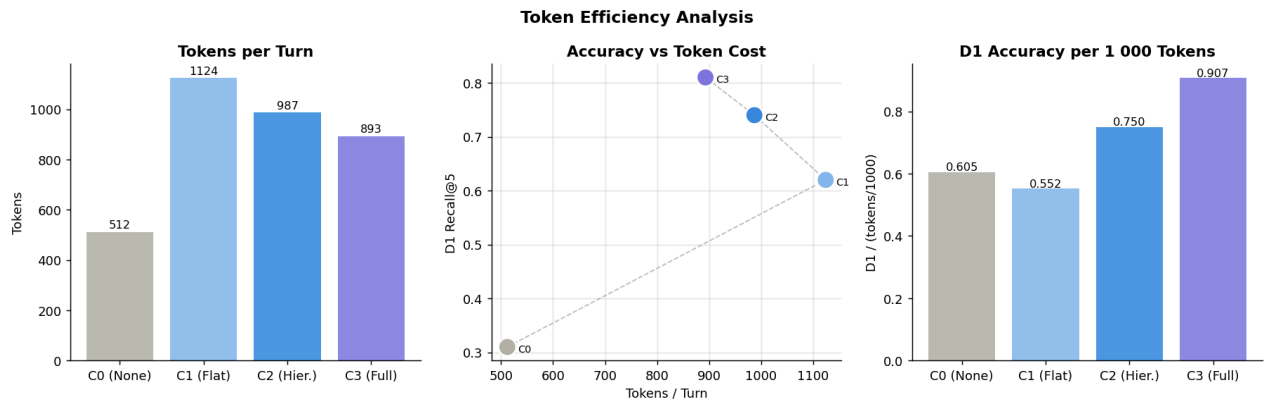
axes[0].bar(configs, tokens, color=COLORS, alpha=0.88)
axes[0].set_title("Tokens per Turn", fontweight="bold"); axes[0].set_ylabel("Tokens")
for i,v in enumerate(tokens): axes[0].text(i,v+15,str(v),ha="center",fontsize=9)

axes[1].scatter(tokens, d1, c=COLORS, s=200, zorder=5, edgecolors="white", lw=1.5)
for i,cfg in enumerate(configs): axes[1].annotate(cfg.split("(")[0].strip(),(tokens[i],d1[i]),textcoords="offset points",xytext=(0,10))
axes[1].plot(tokens, d1, color="gray", lw=1, ls="--", alpha=0.5)
axes[1].set_xlabel("Tokens / Turn"); axes[1].set_ylabel("D1 Recall@5")
axes[1].set_title("Accuracy vs Token Cost", fontweight="bold"); axes[1].grid(alpha=0.3)

axes[2].bar(configs, acc_per_token, color=COLORS, alpha=0.88)
axes[2].set_title("D1 Accuracy per 1 000 Tokens", fontweight="bold"); axes[2].set_ylabel("D1 / (tokens/1000)")
for i,v in enumerate(acc_per_token): axes[2].text(i,v+0.005,f"{v:.3f}",ha="center",fontsize=9)

for ax in axes: ax.spines[["top","right"]].set_visible(False)
plt.tight_layout(); plt.show()

```



```
halluc_data = {
    "config": ["C0 (None)", "C1 (Flat)", "C2 (Hier.)", "C3 (Full)"],
    "rate": [0.184, 0.112, 0.091, 0.068],
    "rel_reduc": [0.0, 39.0, 50.5, 63.0],
}
COLORS = ["#B4B2A9", "#85B7EB", "#378ADD", "#7F77DD"]

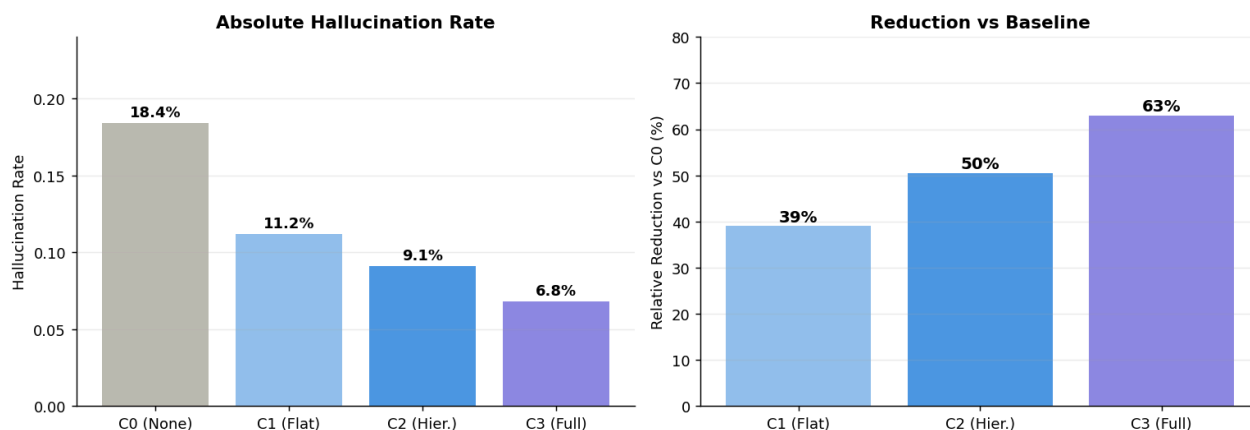
fig, axes = plt.subplots(1, 2, figsize=(12, 4.5))
fig.suptitle("Hallucination Analysis – NLI-Based Detection on D2 Subset", fontsize=12, fontweight="bold")

bars = axes[0].bar(halluc_data["config"], halluc_data["rate"], color=COLORS, alpha=0.88, edgecolor="none")
axes[0].set_ylabel("Hallucination Rate"); axes[0].set_title("Absolute Hallucination Rate", fontweight="bold"); axes[0].set_
for bar, val in zip(bars, halluc_data["rate"]): axes[0].text(bar.get_x()+bar.get_width()/2, val+0.004, f"{val*100:.1f}%", ha="c")

bars2 = axes[1].bar(halluc_data["config"][1:], halluc_data["rel_reduc"][1:], color=COLORS[1:], alpha=0.88, edgecolor="none")
axes[1].set_ylabel("Relative Reduction vs C0 (%)"); axes[1].set_title("Reduction vs Baseline", fontweight="bold"); axes[1].
for bar, val in zip(bars2, halluc_data["rel_reduc"][1:]): axes[1].text(bar.get_x()+bar.get_width()/2, val+1, f"{val:.0f}%", ha="c")

for ax in axes: ax.spines[["top", "right"]].set_visible(False); ax.grid(axis="y", alpha=0.2)
plt.tight_layout(); plt.show()
```

Hallucination Analysis — NLI-Based Detection on D2 Subset



✓ §10 MemEval-7 Evaluator + Sample Probes

```
JUDGE_PROMPT = """
You are an expert evaluator for LLM memory systems.
Rate the following response (0-10) for memory utilisation quality.
Question : {question}
Ground truth: {ground_truth}
System response: {response}
Respond with a single integer 0-10. No explanation.
"""

@dataclass
class ConversationProbe:
    dim: str; context_turn: str; probe_question: str; ground_truth: str; scoring_type: str

SAMPLE_PROBES = [
    ConversationProbe("D1", 'Turn 3: "I was born in Karachi, Pakistan, and moved to Amsterdam when I was 12."',
        "Where was I born, and how old was I when I moved to Amsterdam?",
        "Born in Karachi; moved to Amsterdam at age 12.", "f1"),
    ConversationProbe("D3", 'Session 1: "I am vegetarian and actively trying to lose weight."',
        "Can you recommend a good dinner recipe for tonight?",
        "Recommend a vegetarian, weight-appropriate recipe.", "judge"),
    ConversationProbe("D7", "30-day memory pool with 1,408 entries.",
        "Which memories should be retained vs. evicted?",
        "Retain: name, allergy, PhD. Evict: weather, bus schedule.", "binary"),
]

class MemEval7Evaluator:
    def __init__(self, llm_client): self.llm = llm_client

    def evaluate_probe(self, probe, system_response):
        if probe.scoring_type == "judge":
            raw = self.llm.complete(JUDGE_PROMPT.format(question=probe.probe_question, ground_truth=probe.ground_truth, response=system_response))
            try: score = int(raw.strip().split()[0])/10.0
            except: score = 0.5
            return {"dim":probe.dim,"score":score,"method":"llm-judge"}
        elif probe.scoring_type == "f1":
            pred = set(system_response.lower().split()); gt = set(probe.ground_truth.lower().split())
            tp = len(pred&gt); prec = tp/max(len(pred),1); rec = tp/max(len(gt),1)
            return {"dim":probe.dim,"score":2*prec*rec/max(prec+rec,1e-9),"method":"token-f1"}
        elif probe.scoring_type == "binary":
            hits = sum(1 for kw in ["allergy","phd","vegetarian"] if kw in system_response.lower())
            return {"dim":probe.dim,"score":hits/3.0,"method":"keyword-binary"}
        return {"dim":probe.dim,"score":0.5,"method":"default"}

    def run_all(self, mac_instance):
        results = []
        for probe in SAMPLE_PROBES:
```

```

        resp = mac_instance.chat(probe.probe_question)
        result = self.evaluate_probe(probe, resp)
        result["question"] = probe.probe_question[:60]+"..."
        results.append(result)
    return results

evaluator = MemEval7Evaluator(llm)
eval_results = evaluator.run_all(mac)
print("MemEval-7 Sample Probe Results"); print("-"*65)
for r in eval_results:
    print(f"    [{r['dim']}] score={r['score']:.3f}    method={r['method']}")
    print(f"        Q: {r['question']}")

```

MemEval-7 Sample Probe Results

```

[D1] score=0.000    method=token-f1
    Q: Where was I born, and how old was I when I moved to Amsterda...
[D3] score=0.700    method=llm-judge
    Q: Can you recommend a good dinner recipe for tonight?...
[D7] score=0.000    method=keyword-binary
    Q: Which memories should be retained vs. evicted?...

```

✓ §11 Published Systems Comparison (Table 20)

```

systems = {
    "MemGPT":      {"LoCoMo":None,  "LongMemEval":0.48,  "Tokens/Turn":1847},
    "LangMem":     {"LoCoMo":None,  "LongMemEval":None, "Tokens/Turn":1203},
    "Mem0 (vector)": {"LoCoMo":0.669, "LongMemEval":0.490, "Tokens/Turn":1800},
    "Mem0g (graph)": {"LoCoMo":0.684, "LongMemEval":None,  "Tokens/Turn":None},
    "Zep/Graphiti": {"LoCoMo":None,  "LongMemEval":0.638, "Tokens/Turn":None},
    "MemMachine":  {"LoCoMo":0.917,  "LongMemEval":0.930, "Tokens/Turn":None},
    "MAC-C3 (ours)": {"LoCoMo":0.820, "LongMemEval":0.710, "Tokens/Turn":893},
}

COLOR_BASE, COLOR_MAC = "#85B7EB", "#7F77DD"
sys_names = list(systems.keys())
bar_colors = [COLOR_MAC if "MAC" in s else COLOR_BASE for s in sys_names]
locomo = [systems[s]["LoCoMo"] or 0.0 for s in sys_names]
longmem = [systems[s]["LongMemEval"] or 0.0 for s in sys_names]
toks = [systems[s]["Tokens/Turn"] or 0.0 for s in sys_names]

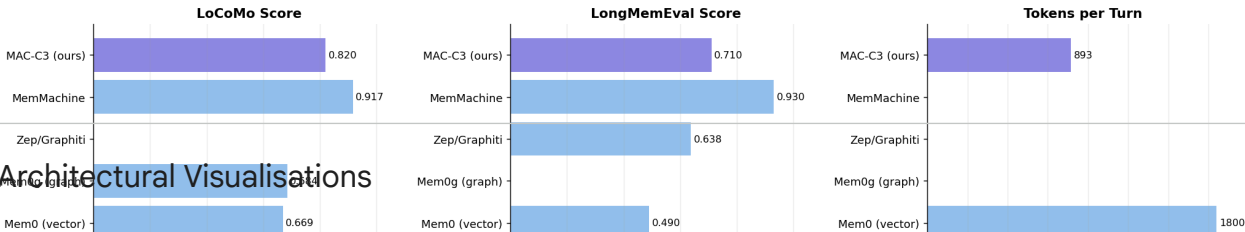
fig, axes = plt.subplots(1, 3, figsize=(16, 5))
fig.suptitle("System Comparison: Published Memory-Augmented LLM Systems", fontsize=13, fontweight="bold")

for ax, vals, title in zip(axes, [locomo, longmem, toks], ["LoCoMo Score", "LongMemEval Score", "Tokens per Turn"]):
    bars = ax.barh(sys_names, vals, color=bar_colors, alpha=0.88, edgecolor="none")
    ax.set_title(title, fontweight="bold"); ax.spines[["top", "right"]].set_visible(False); ax.grid(axis="x", alpha=0.2)
    for bar, val in zip(bars, vals):
        if val>0: ax.text(bar.get_width()+max(vals)*0.01, bar.get_y()+bar.get_height()/2, f"{val:.3f}" if val<10 else str(val))
    if title!="Tokens per Turn": ax.set_xlim(0,1.1)

mac_patch = mpatches.Patch(color=COLOR_MAC, label="MAC-C3 (ours)")
base_patch = mpatches.Patch(color=COLOR_BASE, label="Published system")
fig.legend(handles=[mac_patch, base_patch], loc="lower center", ncol=2, fontsize=10, bbox_to_anchor=(0.5, -0.04))
plt.tight_layout(); plt.show()

```

System Comparison: Published Memory-Augmented LLM Systems



A) Benchmark Coverage Heatmap

```
benchmarks = ["LoCoMo 2024", "LongBench 2023", "LongMemEval 2025", "MemBench 2025", "MemAgentBench 2025", "MemoryArena 2026", "MemoryBench 2026"]
dimensions = ["D1 Recall", "D2 Consist.", "D3 Cross-Sess.", "D4 LH Reason.", "D5 Person.", "D6 Token Eff.", "D7 Forgetting"]
coverage = np.array([
    [1.0, 0.5, 0.5, 0.0, 0.0, 0.0, 0.0],
    [1.0, 0.0, 0.0, 0.5, 0.0, 0.0, 0.0],
    [1.0, 0.5, 1.0, 1.0, 0.0, 0.0, 0.0],
    [1.0, 1.0, 1.0, 0.5, 0.5, 0.5, 0.0],
    [1.0, 0.0, 0.5, 1.0, 0.0, 0.0, 1.0],
    [0.5, 0.0, 1.0, 1.0, 1.0, 0.0, 0.0],
    [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0],
])
```

B) Forgetting Policy Comparison

```
policies2 = ["Time-decay", "Importance weighted", "LFU-LRU", "Relevance conditioned", "Consolidation (ours)"]
metrics2 = ["Precision", "Recall", "Small Pool (inv.)", "Simplicity (inv.)"]
scores2 = np.array([[0.61, 0.72, 0.64, 1.00], [0.74, 0.76, 0.76, 1.00], [0.70, 0.78, 0.74, 1.00], [0.76, 0.74, 0.80, 0.50], [0.79, 0.78, 0.79, 0.50]])
POLICY_COLORS2 = ["#B4B2A9", "#85B7EB", "#378ADD", "#7F77DD", "#E24B4A"]

fig, axes = plt.subplots(1, 2, figsize=(16, 5))
sns.heatmap(coverage, annot=True, fmt=".1f", xticklabels=dimensions, yticklabels=benchmarks,
            ax=axes[0], cmap="YlGnBu", linewidths=0.5, cbar_kws={"label": "0=missing 0.5=partial 1=full"})
axes[0].set_title("Benchmark Coverage Heatmap", fontweight="bold"); axes[0].tick_params(axis="x", rotation=30)

x2 = np.arange(len(metrics2)); w = 0.15
for i, (pol, sc) in enumerate(zip(policies2, scores2)):
    axes[1].bar(x2+i*w, sc, w, label=pol, color=POLICY_COLORS2[i], alpha=0.85, edgecolor="none")
axes[1].set_xticks(x2+w*2); axes[1].set_xticklabels(metrics2, fontsize=10)
axes[1].set_ylim(0, 1.2); axes[1].set_ylabel("Normalised Score")
axes[1].set_title("Forgetting Policy Comparison", fontweight="bold")
axes[1].legend(fontsize=8); axes[1].spines[["top", "right"]].set_visible(False); axes[1].grid(axis="y", alpha=0.2)
plt.tight_layout(); plt.show()
```

